

인공지능을 활용한 양극소재 기반 배터리 특성 예측

05 결정구조 특성 분류를 위한 딥러닝 모델 이해와 적용

목차

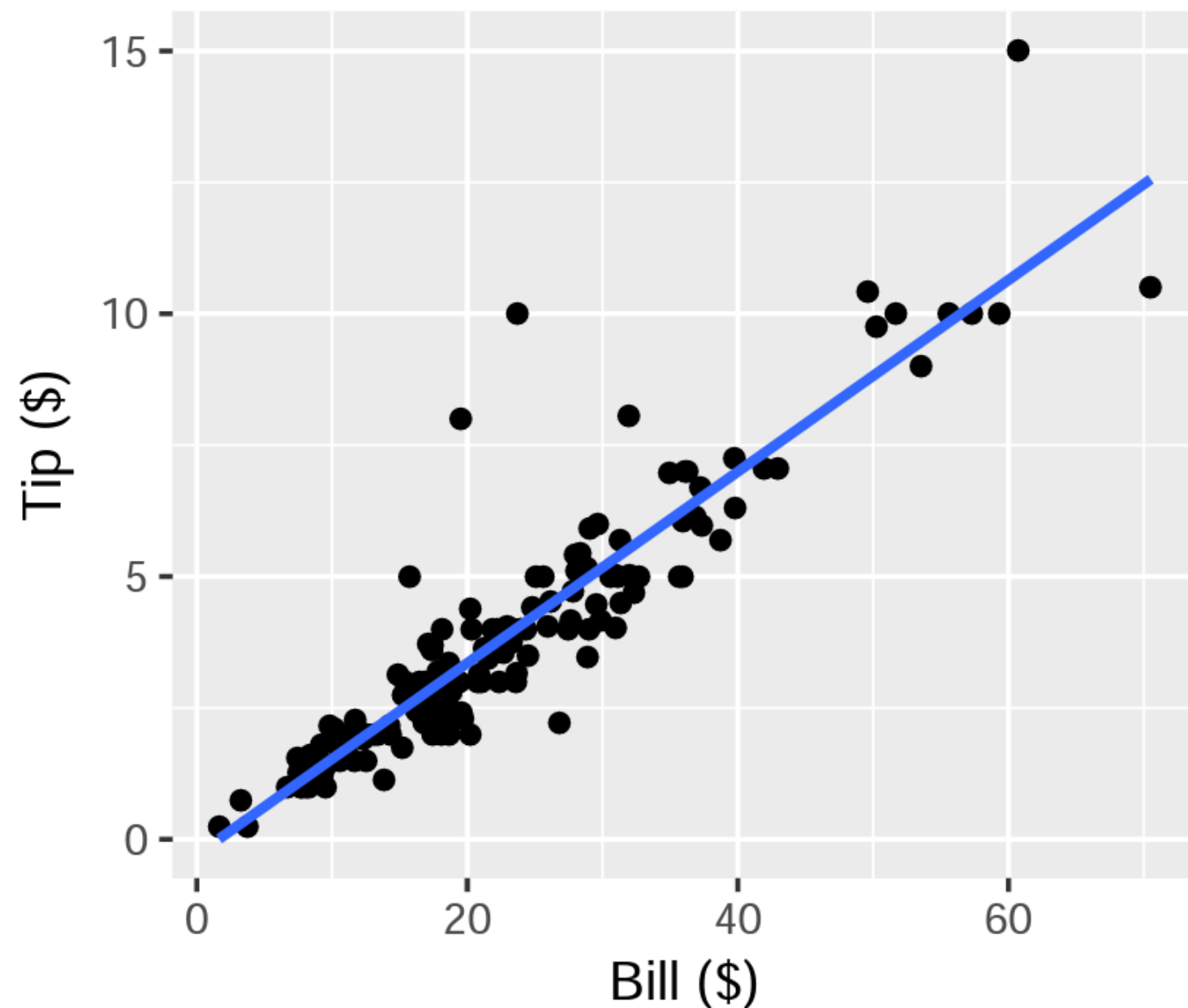
—
결정구조 특성 분류를
위한 딥러닝 모델 이해와
적용

- 01 선형과 비선형 데이터
- 02 퍼셉트론
- 03 다층 퍼셉트론(MLP) 모델
- 04 원-핫 인코딩
- 05 소프트맥스
- 06 옵티마이저

01

선형과 비선형 데이터

☑ 선형 데이터

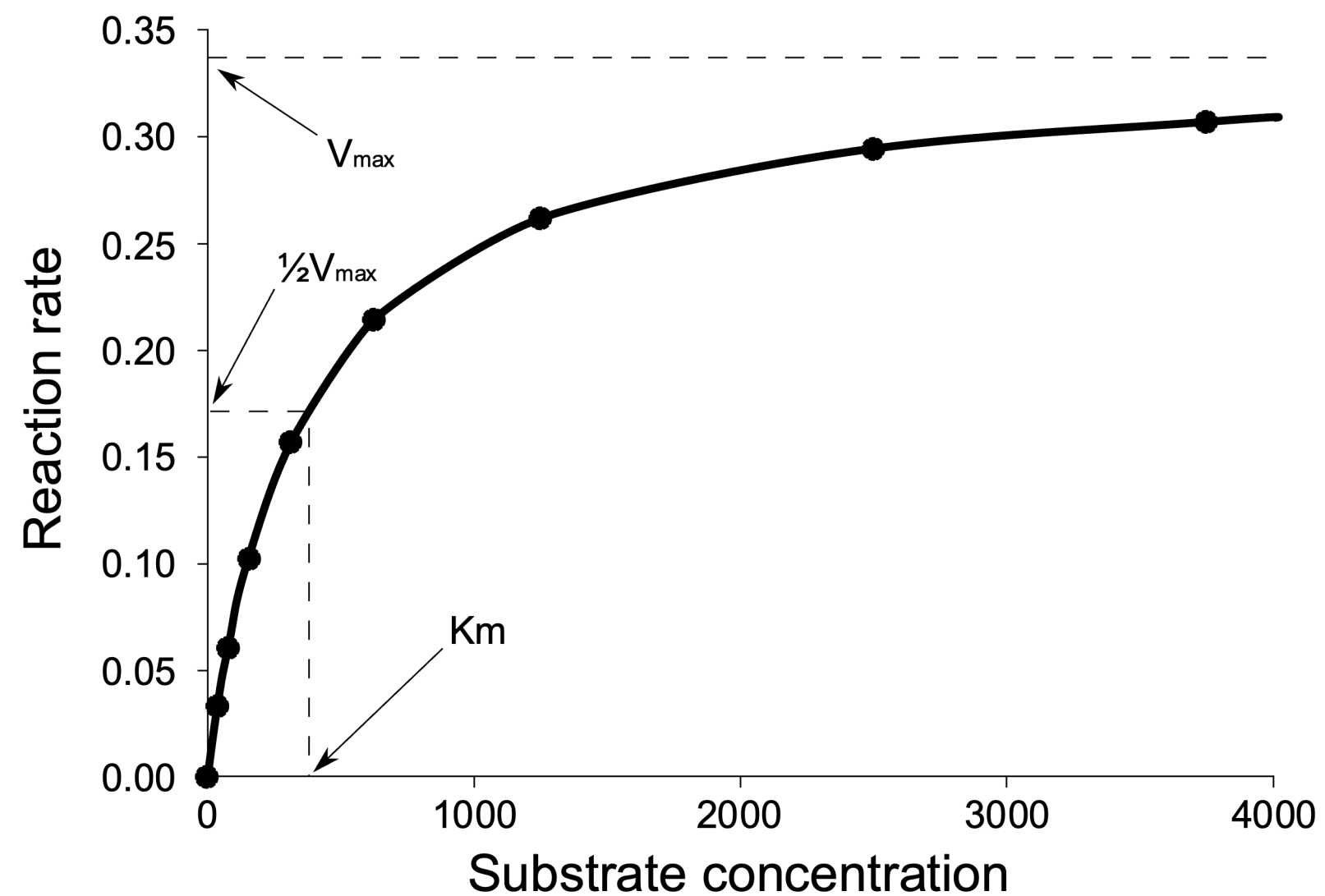


- 독립변수와 종속변수 사이에는 선형 관계가 있을 때
- 독립변수의 **선형 조합**으로 종속변수를 구성할 수 있음

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 \dots + \beta_n X_n$$

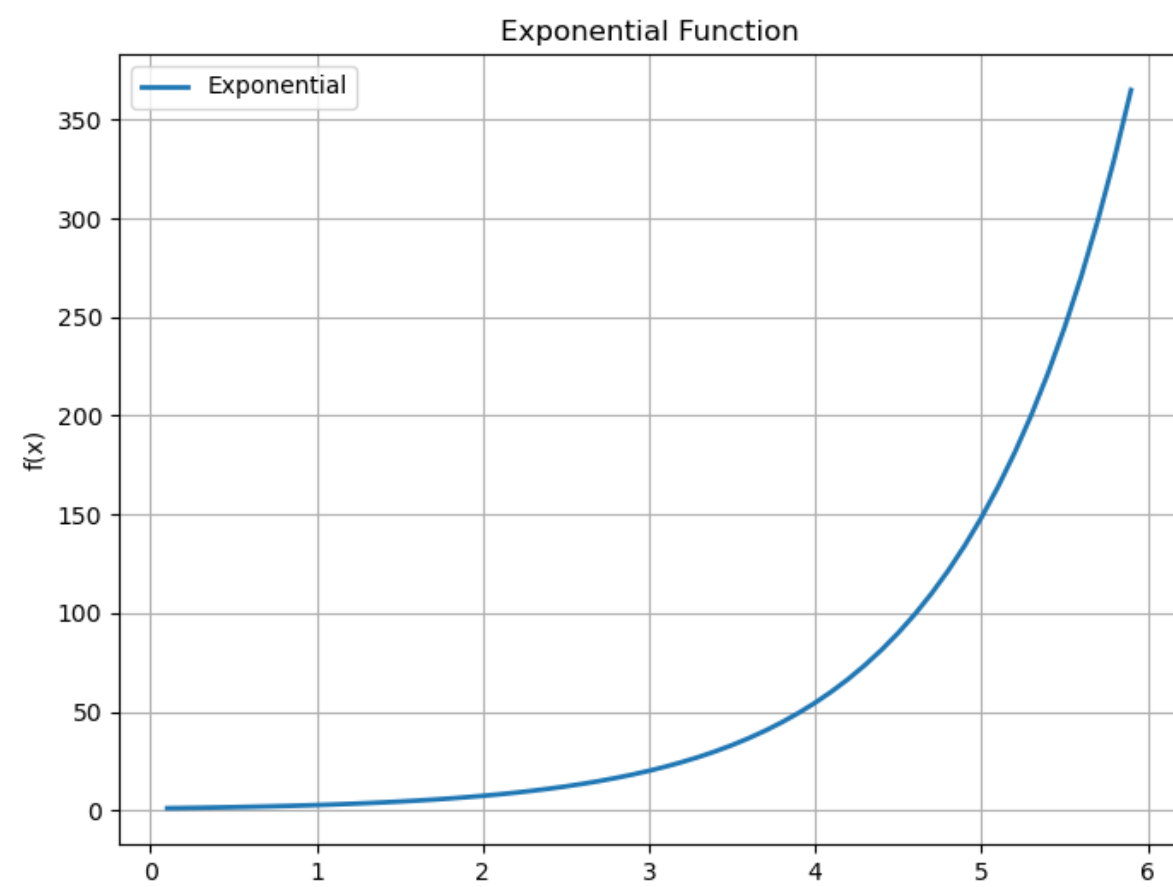
- 일반적으로 하나의 독립변수와 하나의 종속변수를 그래프 시각화했을 때, 직선으로 표현이 가능함
- 비선형성이 매우 작을 때 선형성을 가정할 수 있음
- 예측 알고리즘의 복잡도가 크게 감소함

✓ 비선형 데이터



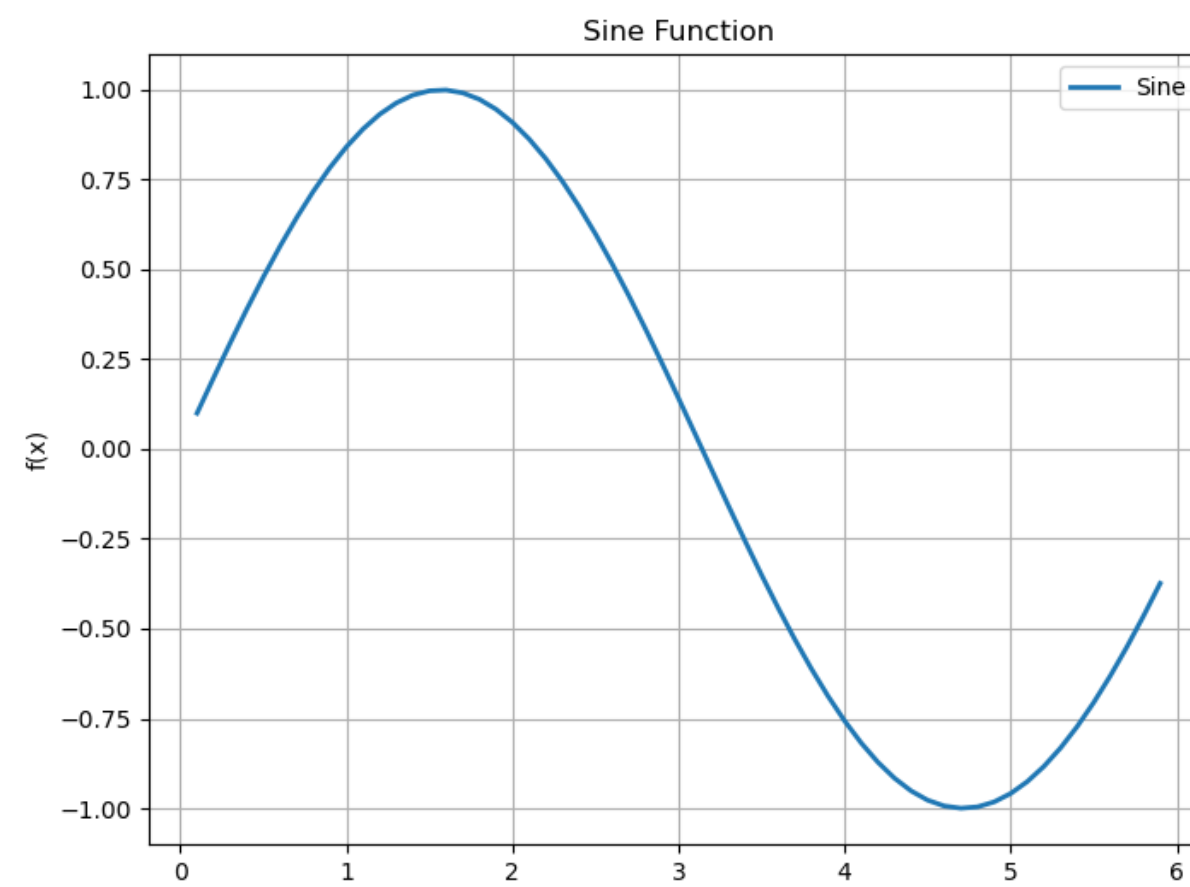
- 독립변수와 종속변수 사이에는 비선형 관계가 있을 때
- 현실에서 자연적으로 발생하는 대부분의 데이터는 비선형성을 띄고 있음
- 고차원 데이터에서는 특히 비선형 관계로 이뤄진 경우가 많음
- 비선형 데이터를 일부 작은 구간에 한정적으로 선형화하기도 함

☑ 대표적인 비선형 함수



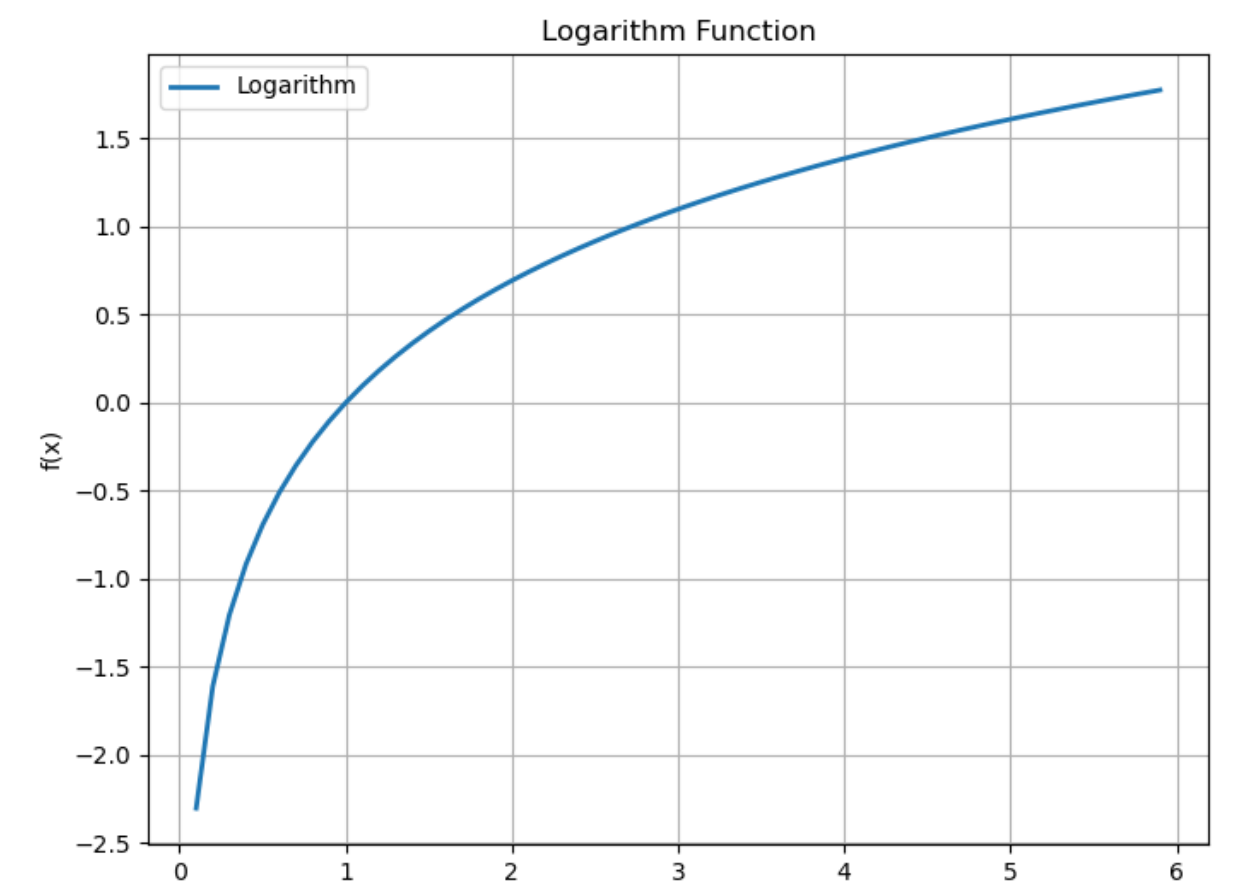
지수 함수

$$f(x) = e^{-x}$$



삼각 함수

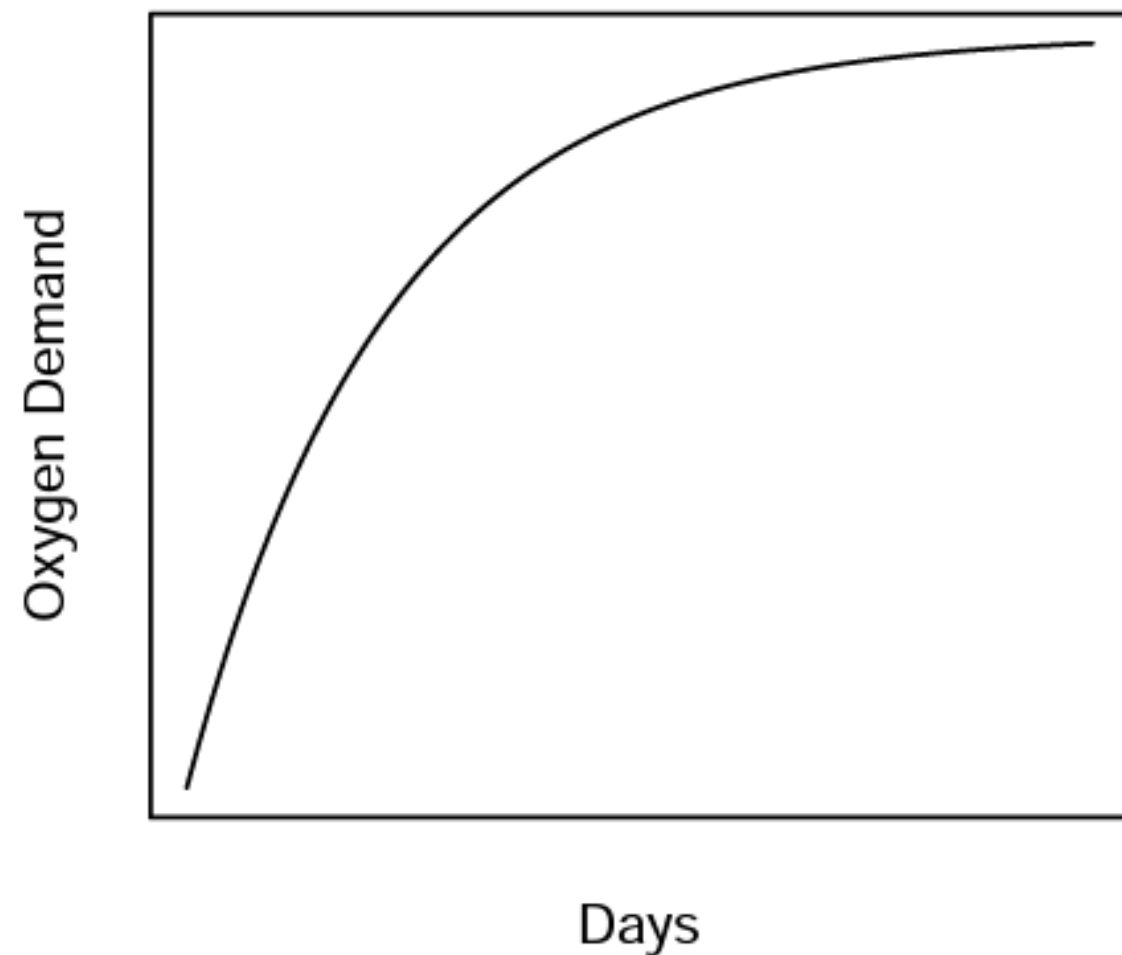
$$f(x) = \sin x$$



로그 함수

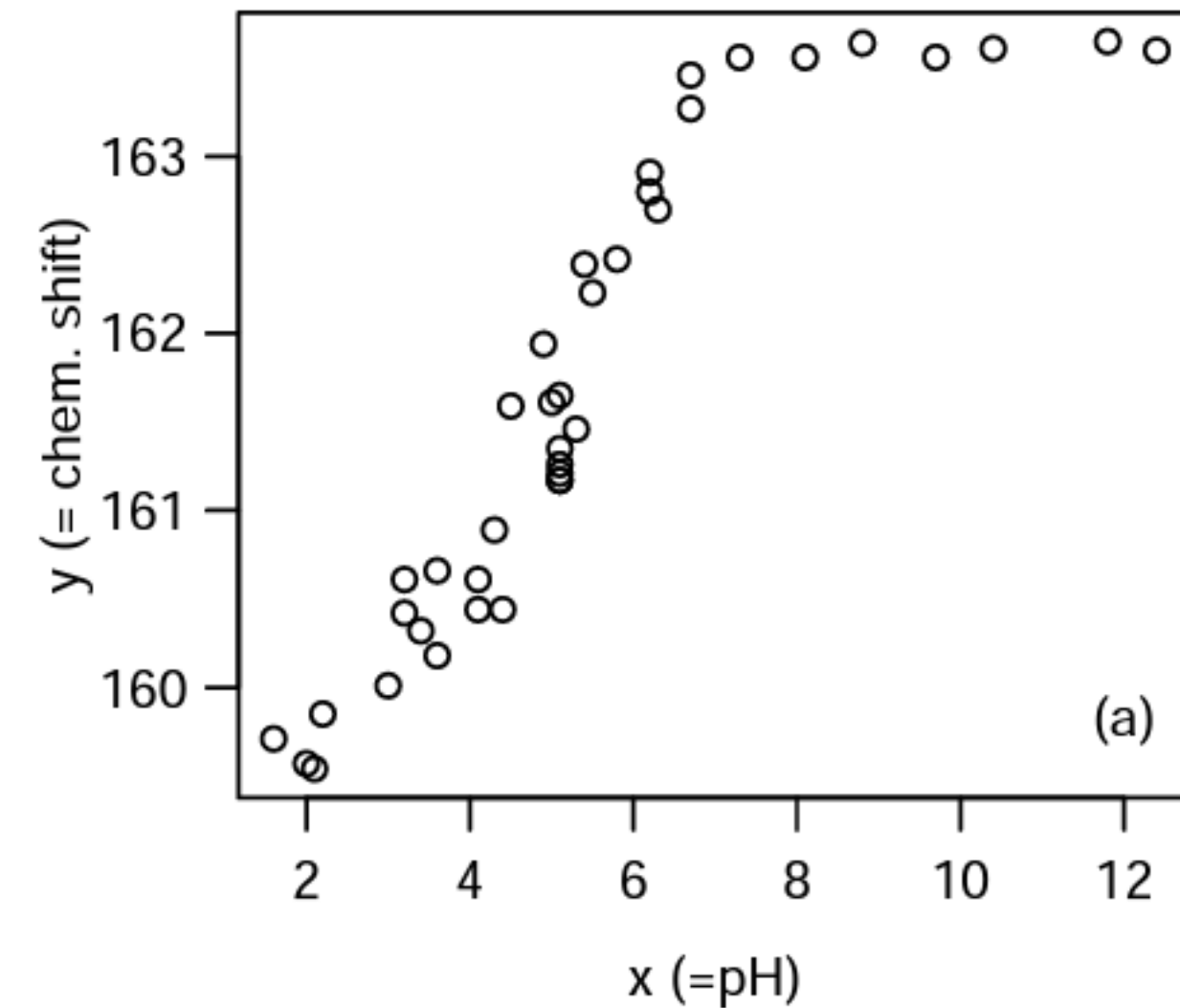
$$f(x) = \log(x)$$

☑ 비선형 데이터 예시



산소 소비량:

미생물의 생물화학적 산소 소비량



화학공학의 멤브레인 분리 기술:

외부 용액 pH값에 따라 변하는 셀룰로우스 막 기공 내 양성자

화된 카복실기

☑ 비선형 데이터 선형화 예시

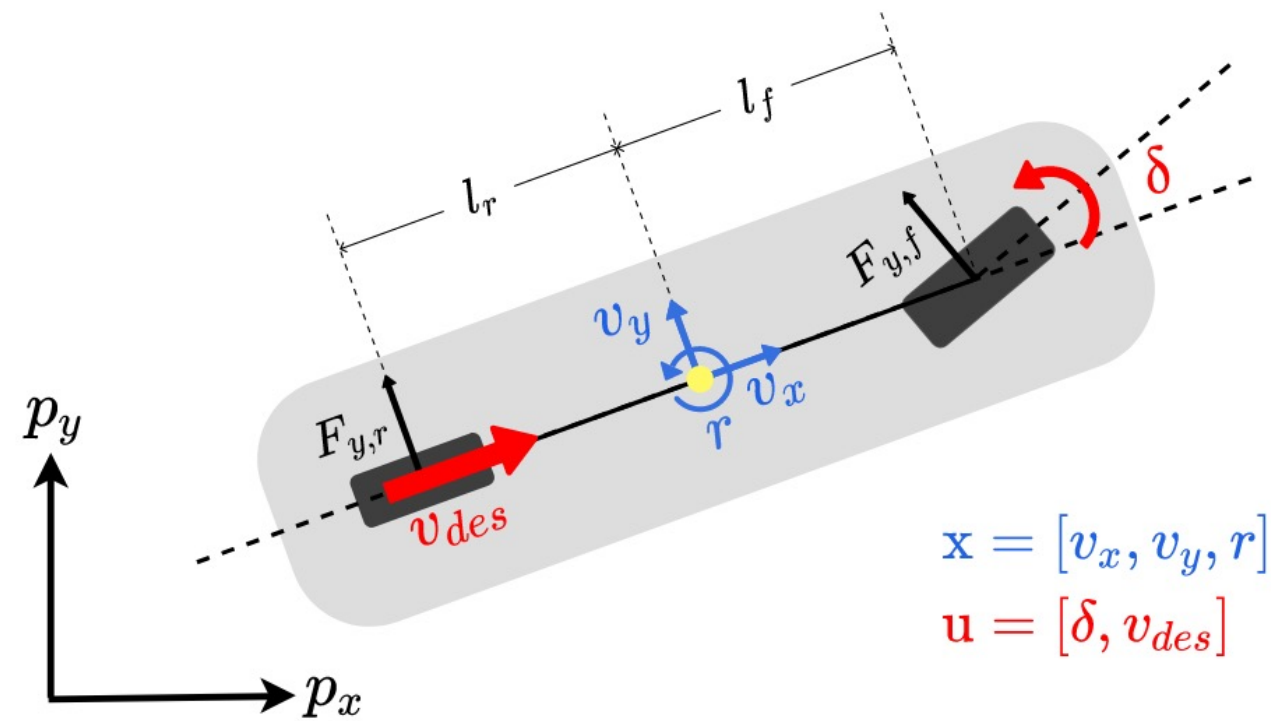
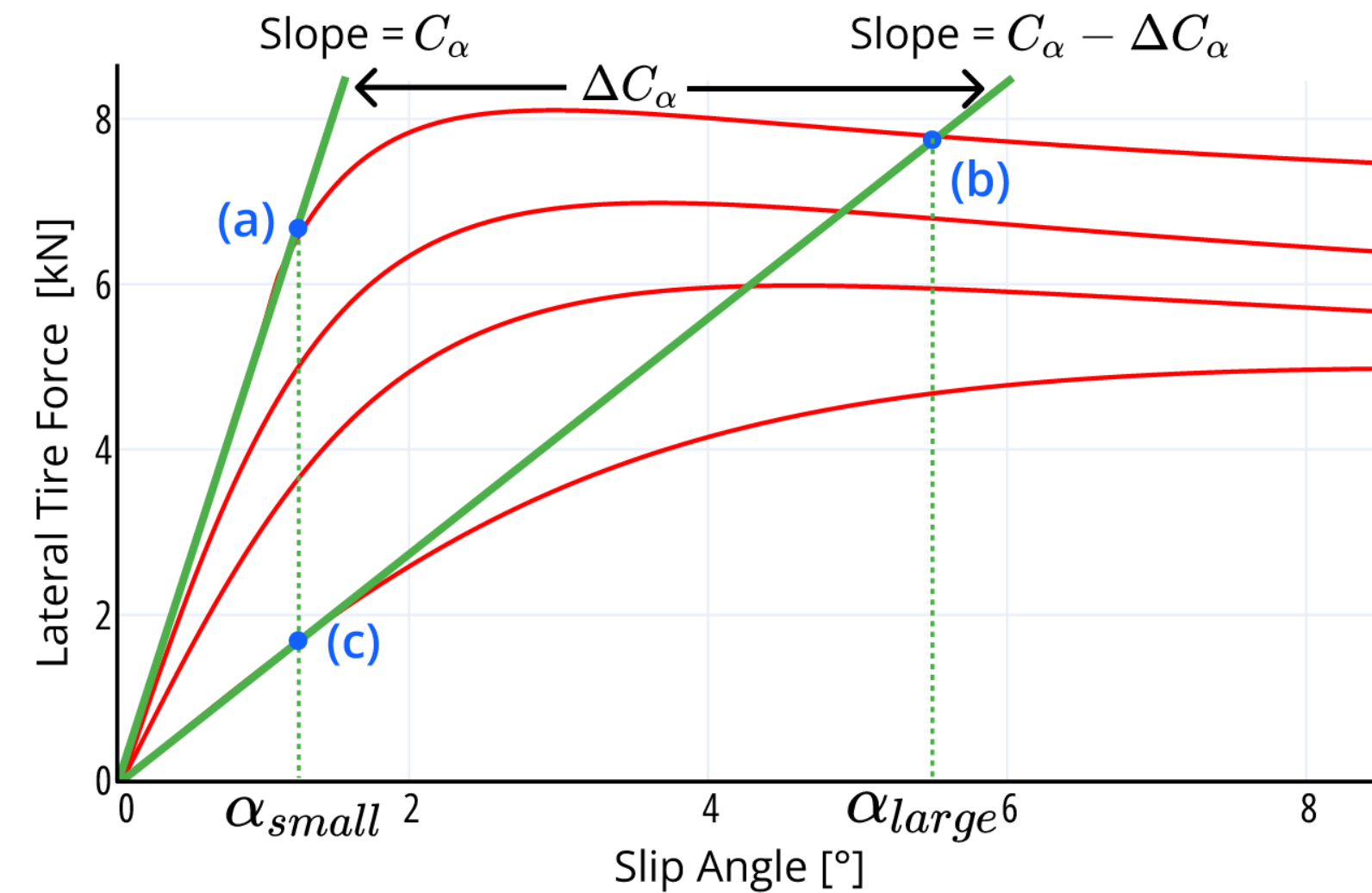


Fig. 2: A diagram of the bicycle model. The dynamic state variable $\mathbf{x}$ is depicted in blue. The input variable $\mathbf{u}$ is depicted in red.

차량의 타이어 모델에서 힘과 슬립각 사이에는 큰 비선형성을 띤

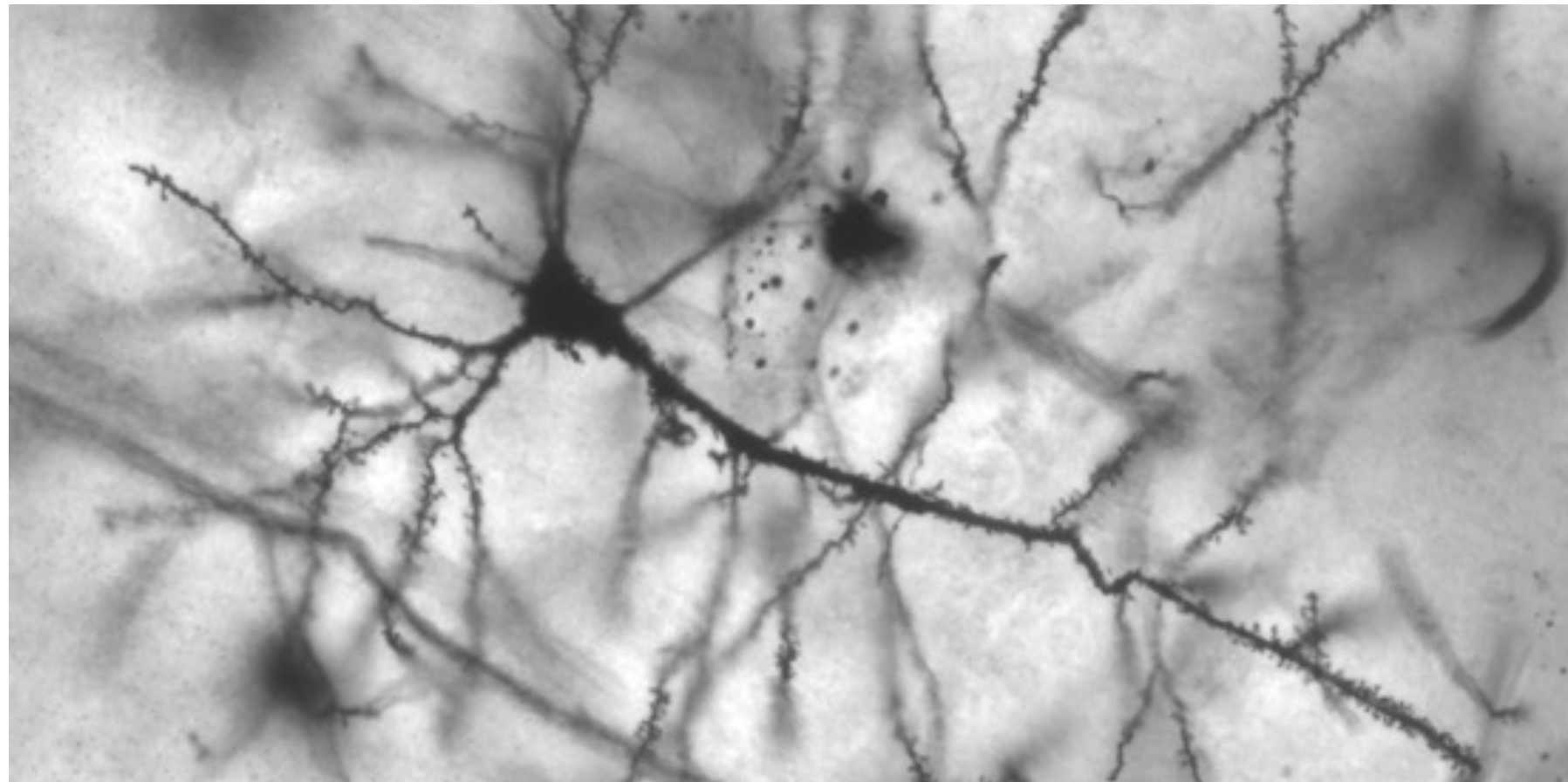


매우 저속에서 미끄럽지 않은 노면을 주행한다는 가정 하에,
매우 작은 슬립각 구간을 선형화할 수 있음

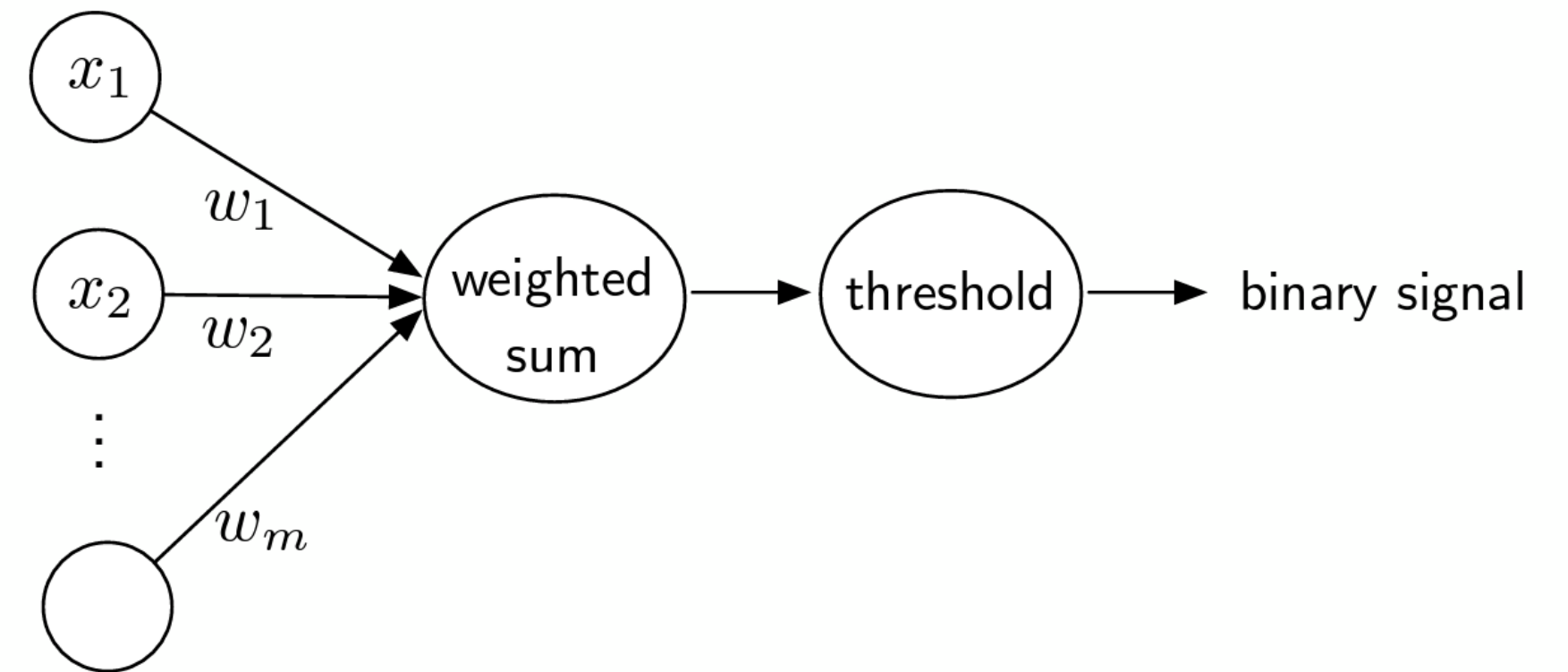
02

퍼셉트론

✓ 퍼셉트론과 뉴런



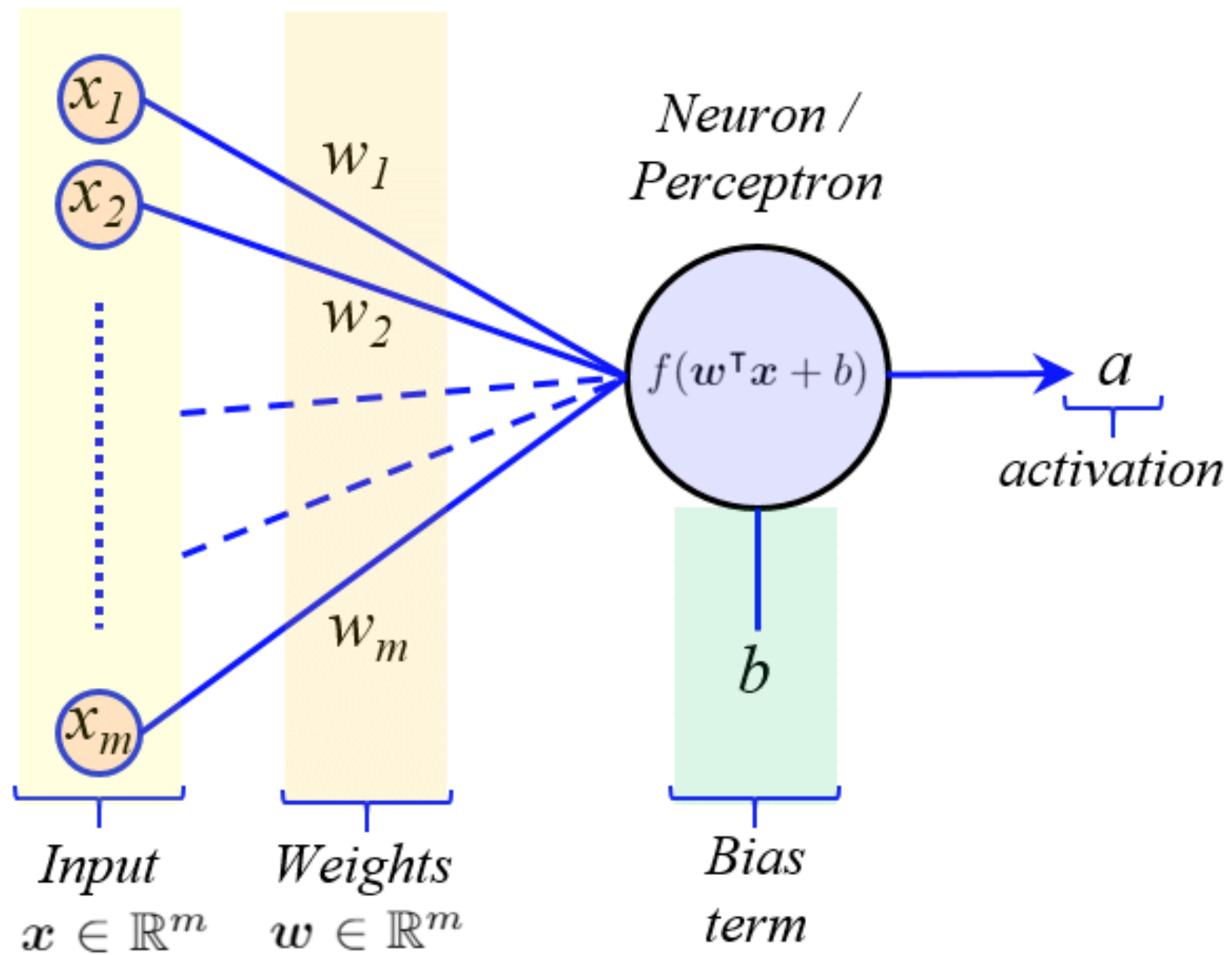
생물학적 뇌와 뉴런



초기 뉴런 모델

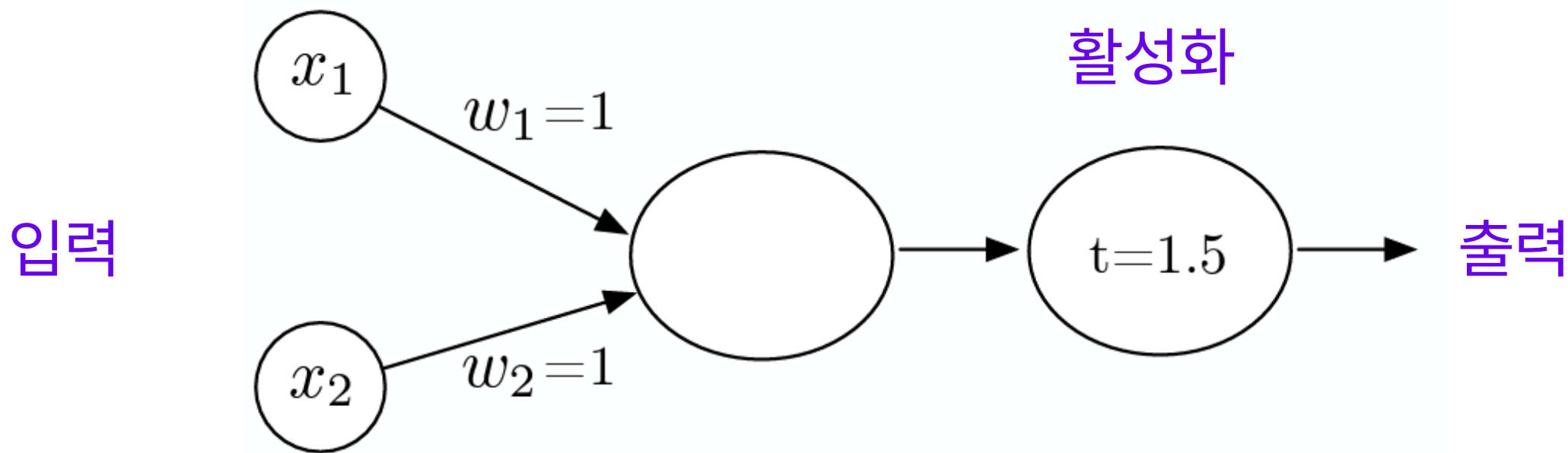
"A Logical Calculus of the Ideas Immanent in Nervous Activity", 1943

④ 퍼셉트론의 기본 구조



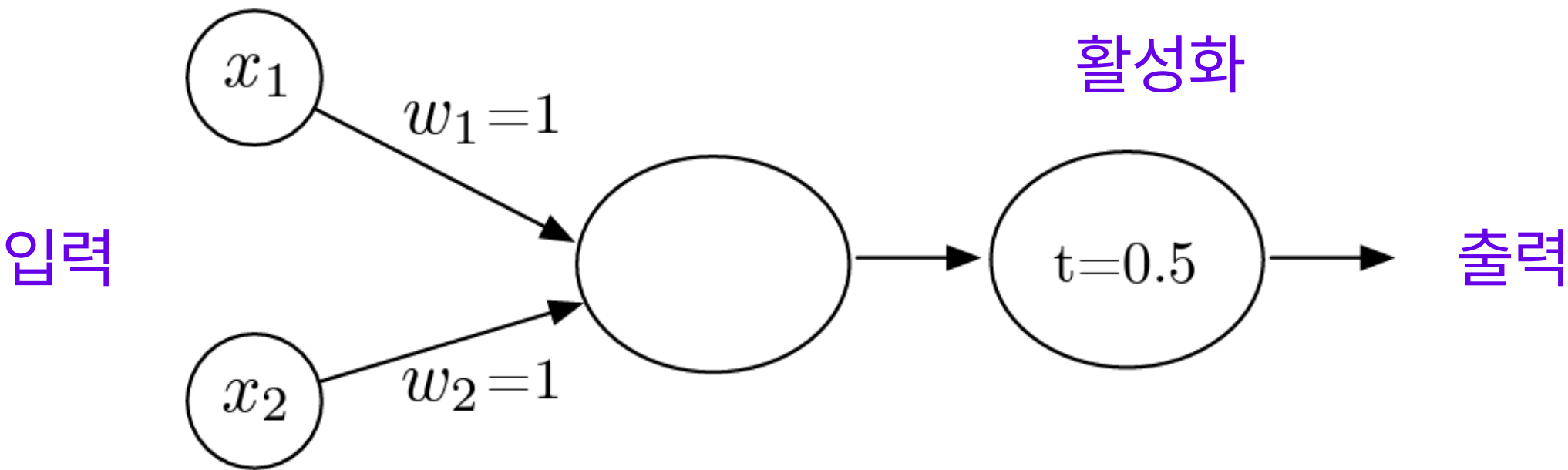
- 입력 변수와 일대일 매칭되는 가중치(Weight)
- 입력변수와 가중치는 곱해짐
- 하나의 편향치(Bias)는 더해짐
- 활성화 함수(Activation Function)을 거쳐 출력됨
- 퍼셉트론만으로 논리 회로를 설계할 수 있음

AND 게이트



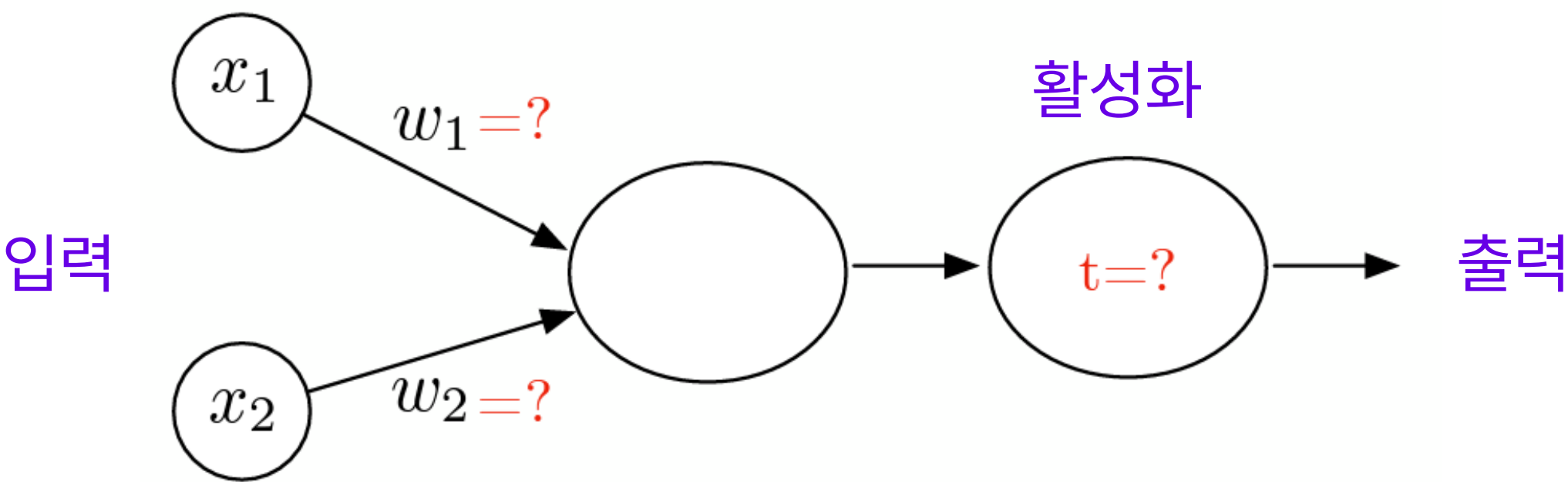
x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

OR 게이트



x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

☑ XOR 게이트

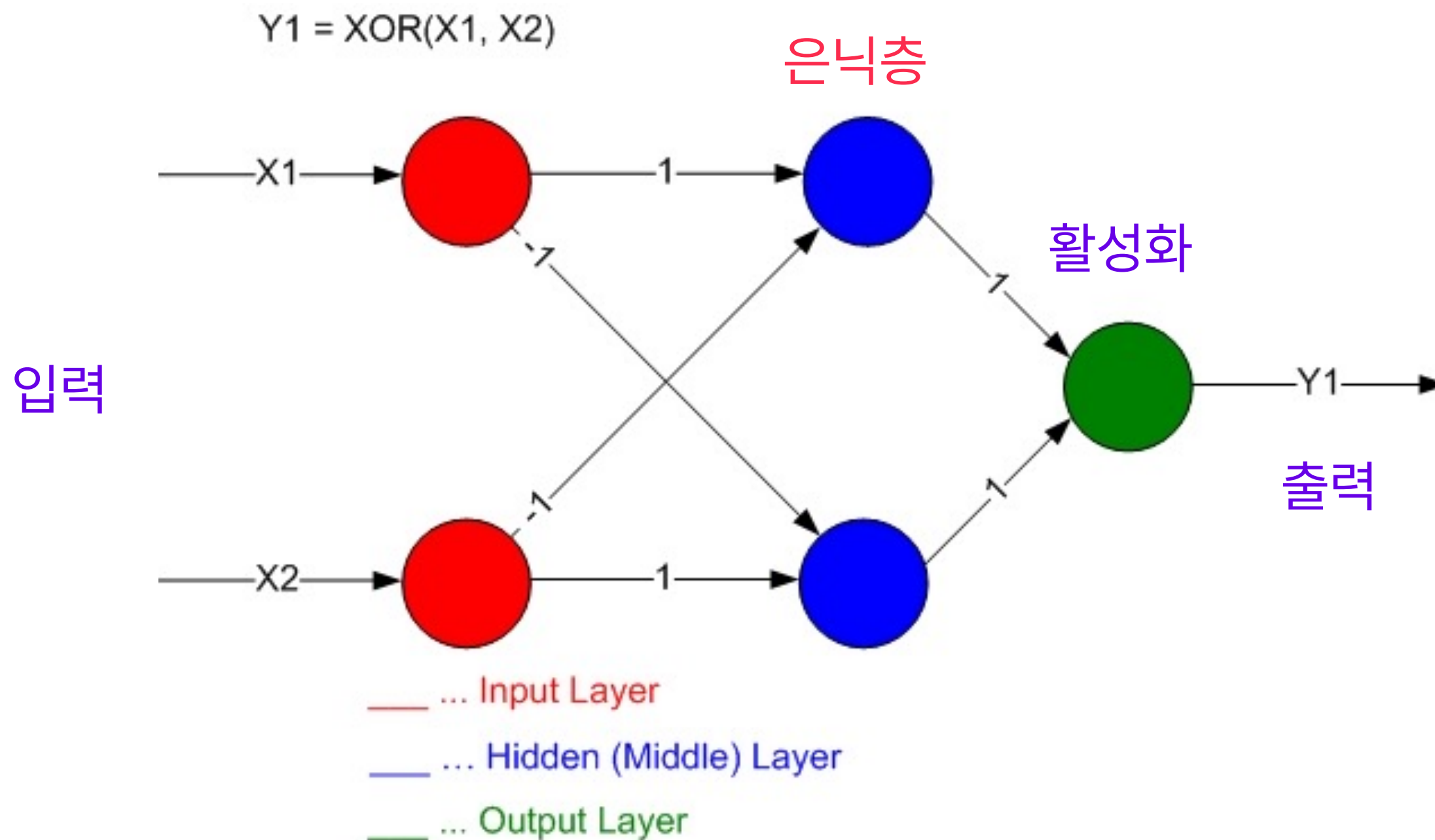


x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

03

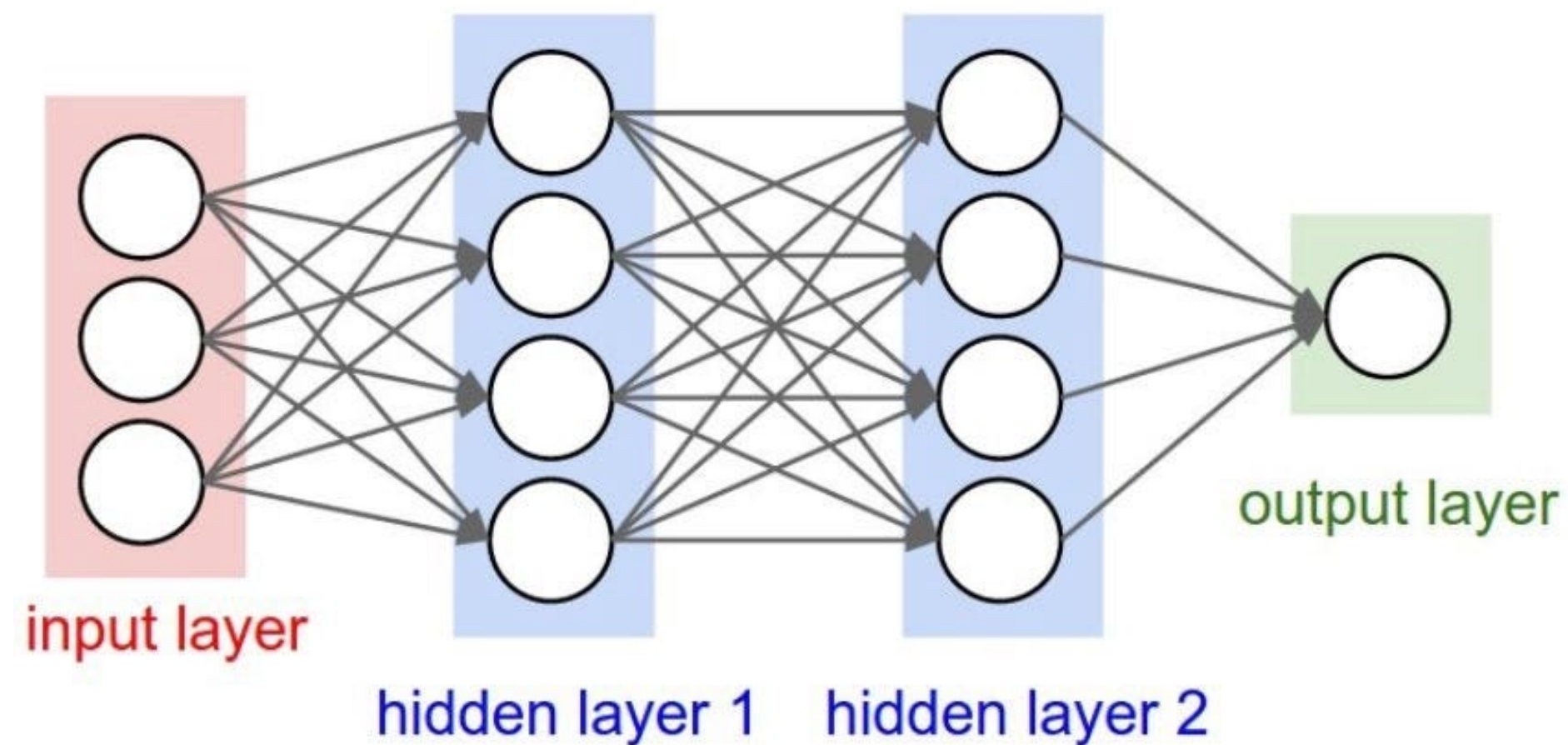
다층 퍼셉트론(MLP) 모델

☑ XOR 게이트



- 여러개의 퍼셉트론을 쌓아서 XOR 게이트 표현 가능
- 입력을 받는 뉴런: 입력층
- 출력하는 뉴런: 출력층
- 그 외의 뉴런: 은닉층(Hidden Layer)

④ 다층 퍼셉트론(MLP)



- 퍼셉트론을 층으로 쌓아서 Multi-Layer Perceptron (MLP)라고 부름
- 가중치와 편향치, 활성화 함수로 이뤄지며 퍼셉트론과 기본 구조는 동일함
- 활성화 함수를 제외하면 선형적인 모델링이며, 곱하기와 더하기로만 이뤄짐

☑ 대표적인 비선형 모델

결정 트리

- 랜덤 포레스트(Random Forest)
- 조건부 추론 트리 (Conditional Inference Tree)

그래디언트 부스팅

- XGBoost
- LightGBM
- CatBoost

인공신경망 (ANN)

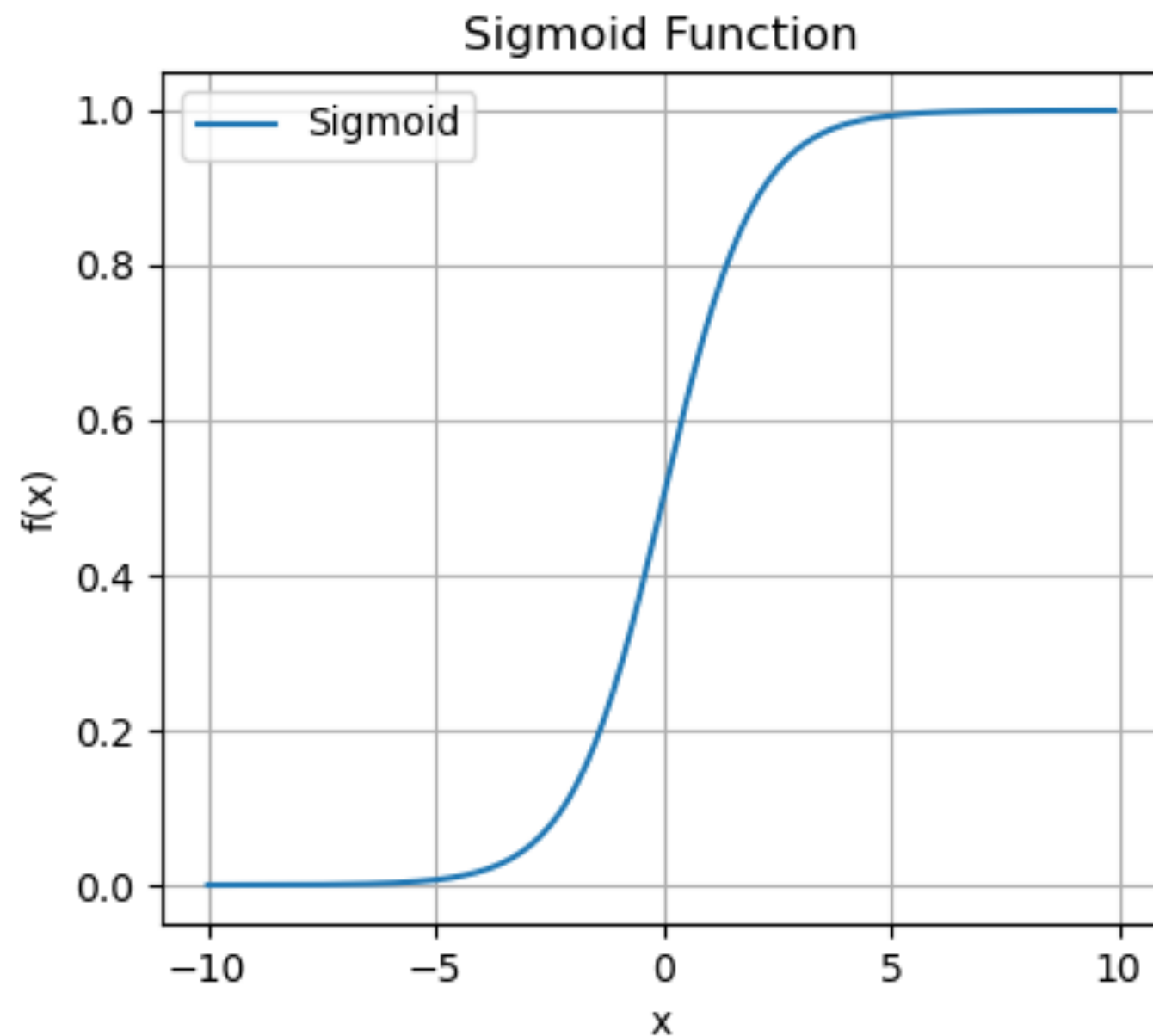
- 다층 퍼셉트론(MLP)
- 합성곱 신경망(CNN)
- 순환 신경망(RNN)
- 스파이크 신경망(SNN)



용어

- XGBoost:** Extreme Gradient Boosting
- ANN:** Artificial Neural Network
- CNN:** Convolutional Neural Network
- RNN:** Recurrent Neural Network
- SNN:** Spike Neural Network

④ 활성화 함수 (1) – 시그모이드(Sigmoid)

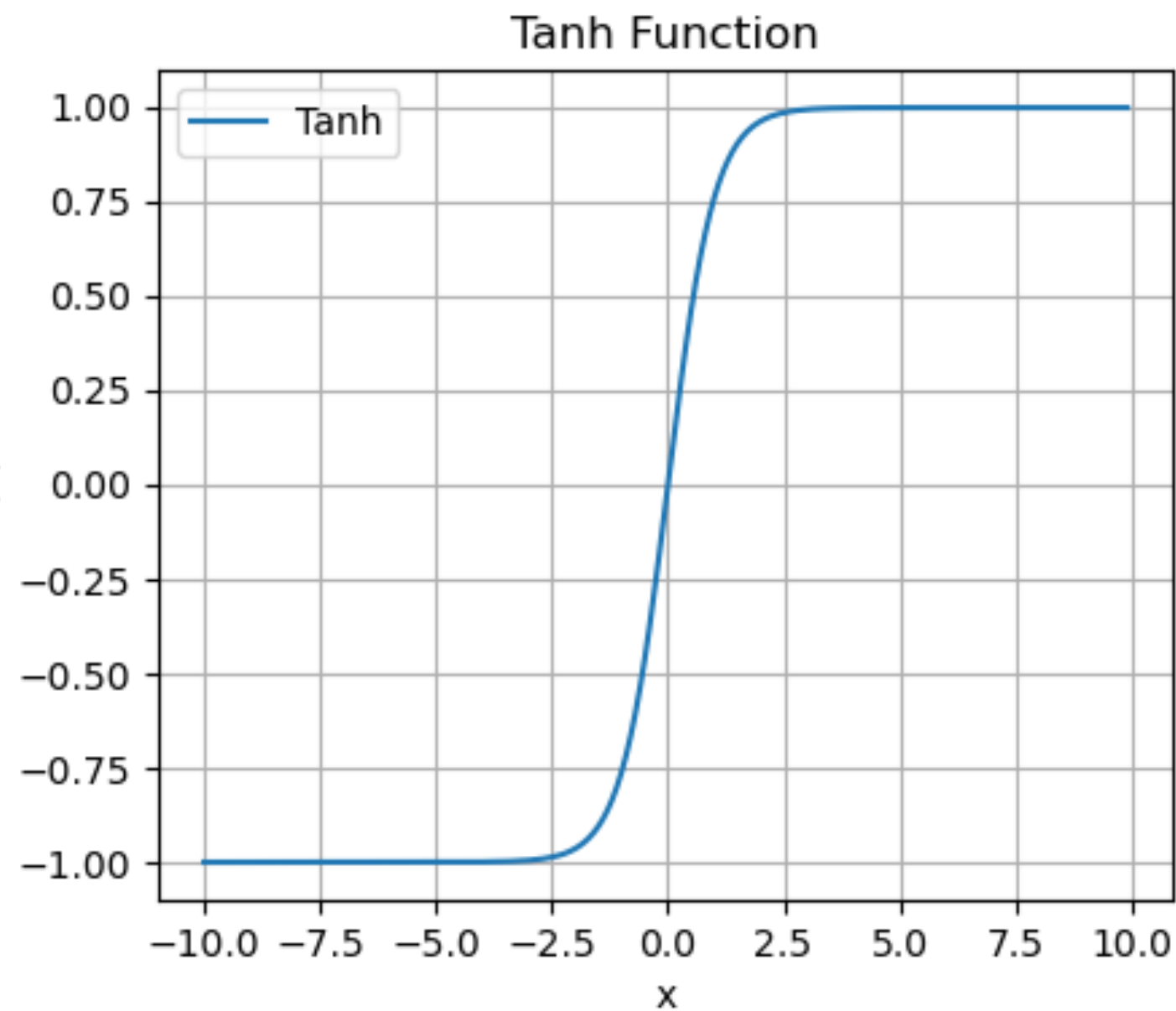


- MLP 모델의 비선형성은 활성화 함수에서 생김
- 비선형적인 표현력을 높이려면 은닉층을 추가해야 함
- 초기의 활성화 함수 중 하나인 시그모이드

$$f(x) = \frac{1}{1 + e^{-x}}$$

- 로지스틱 회귀와 이진 분류에서 사용되기도 함

④ 활성화 함수 (2) – 하이퍼볼릭 탄젠트(Tanh)

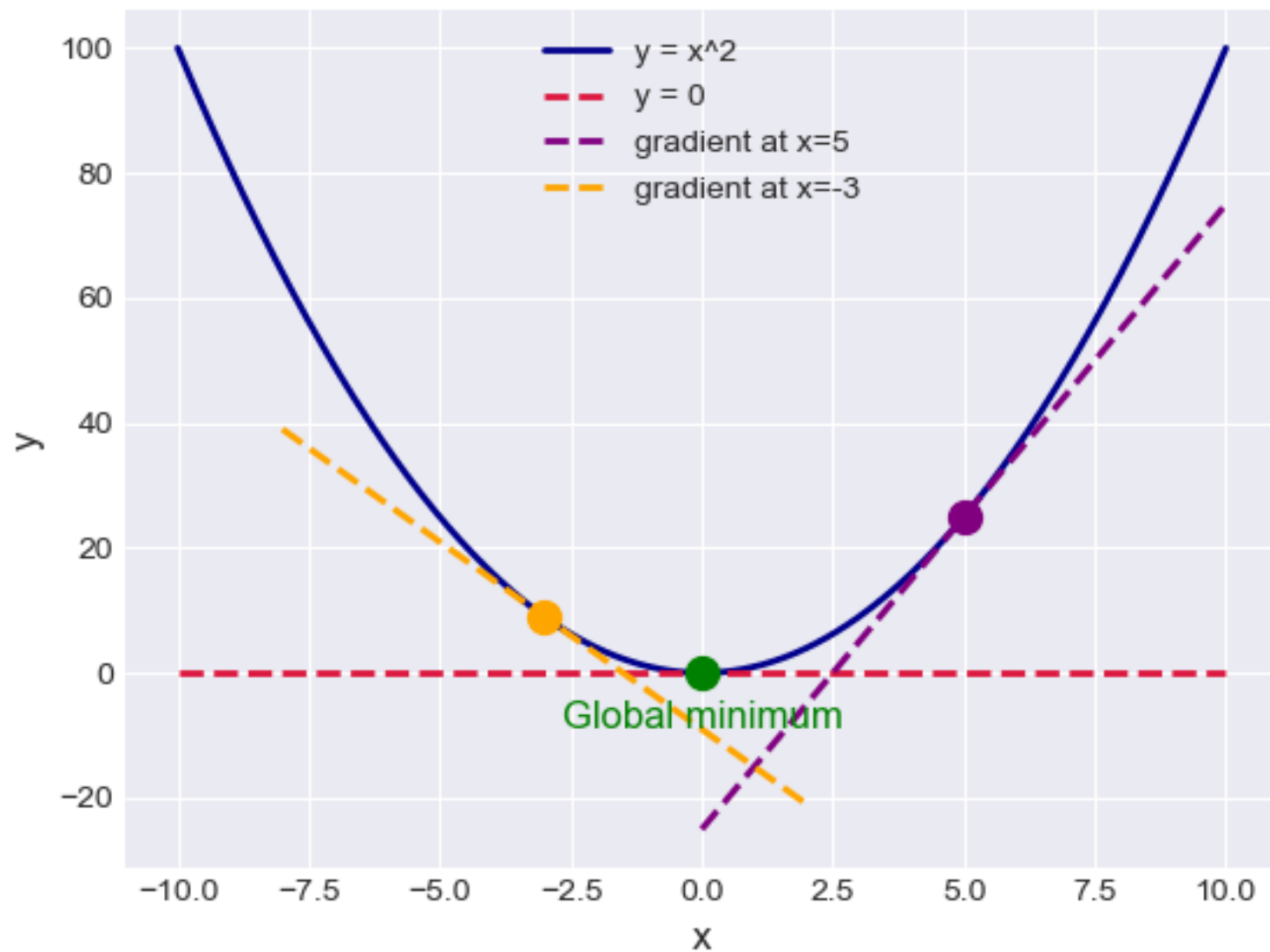


- 시그모이드와 유사한 비선형 활성화 함수
- 시그모이드와 달리 출력의 중심이 0

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

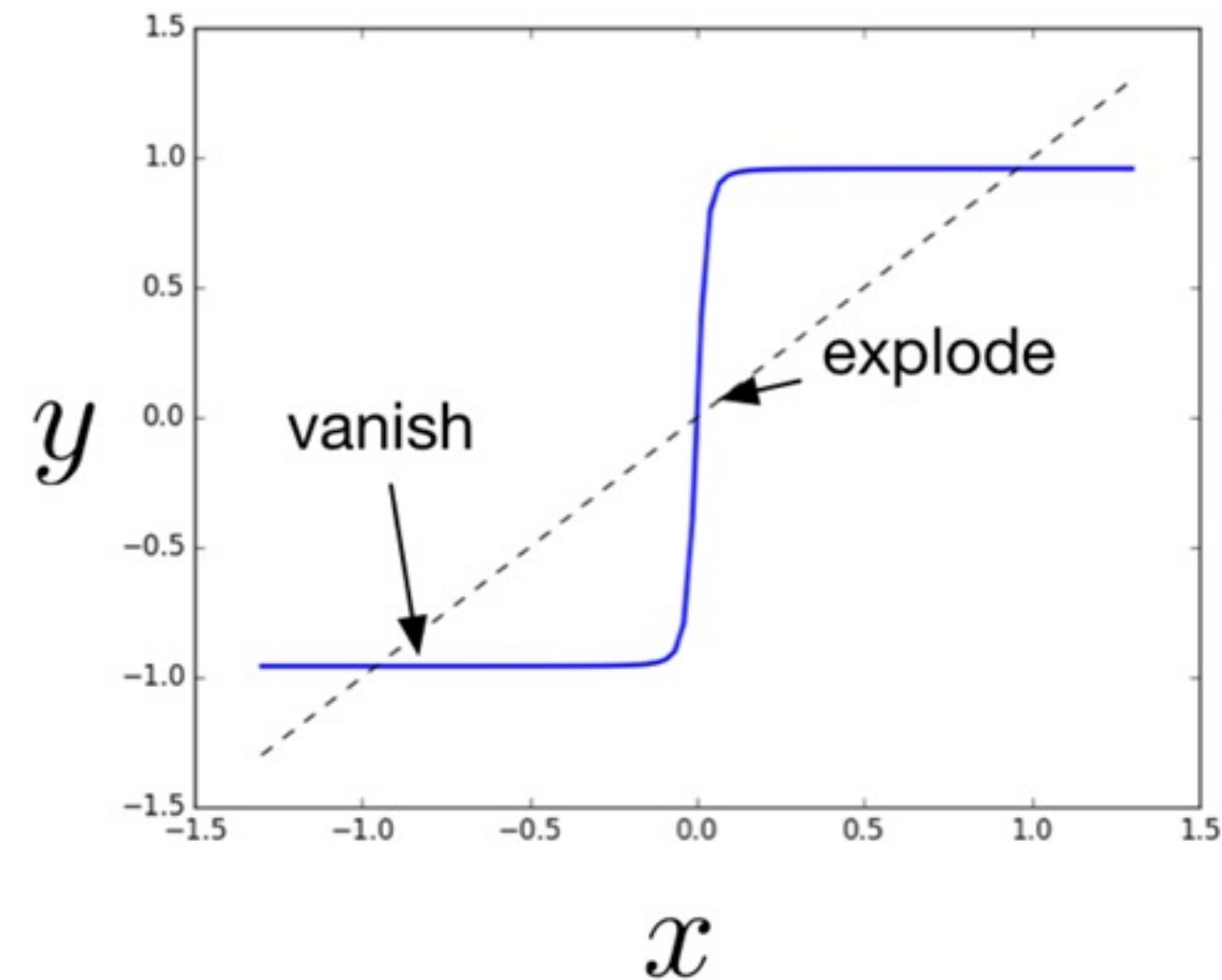
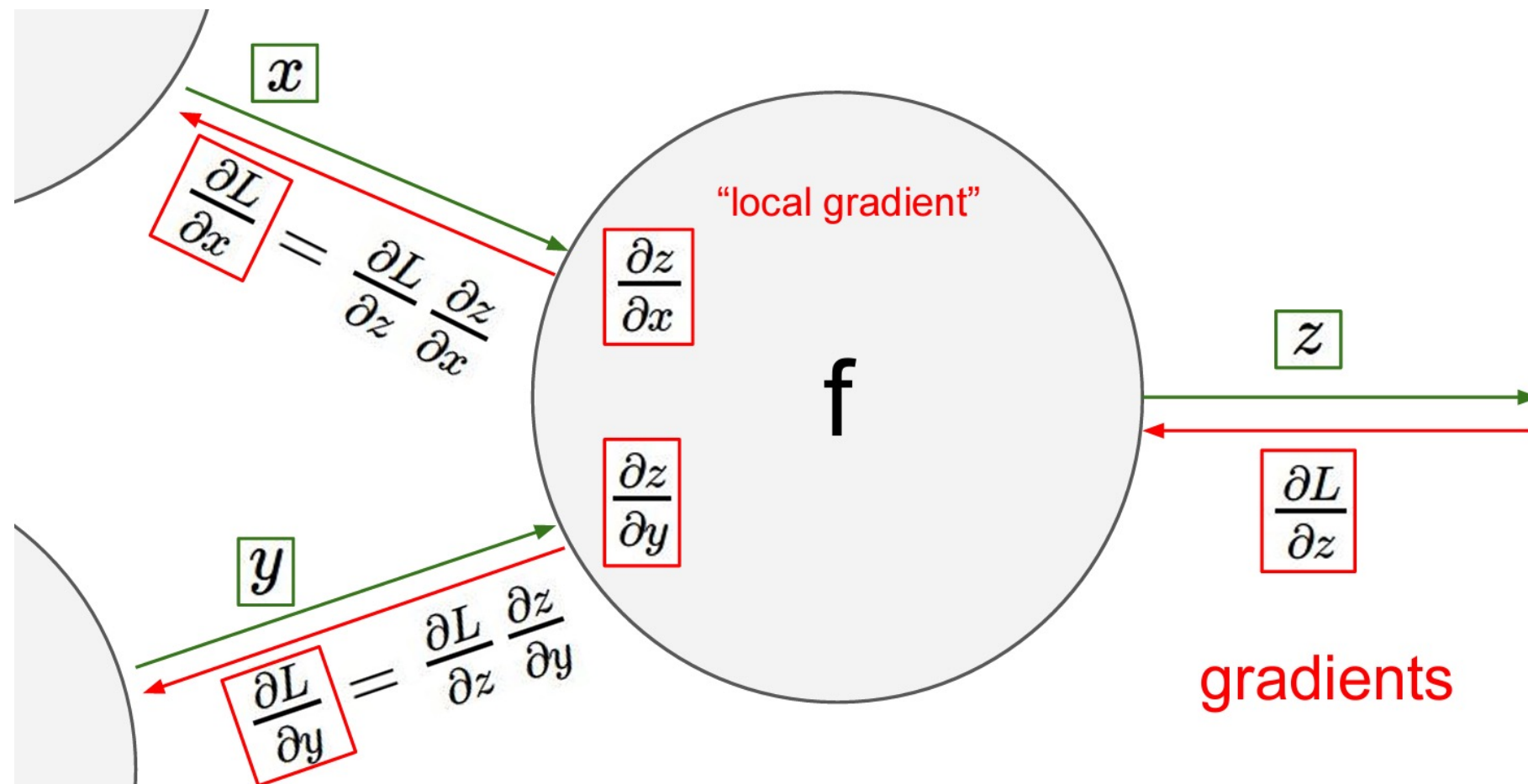
- 두 활성화 함수 모두 기울기 소실(Gradient Vanishing) 문제가 있음

☑ 경사 강하법(Gradient Descent)



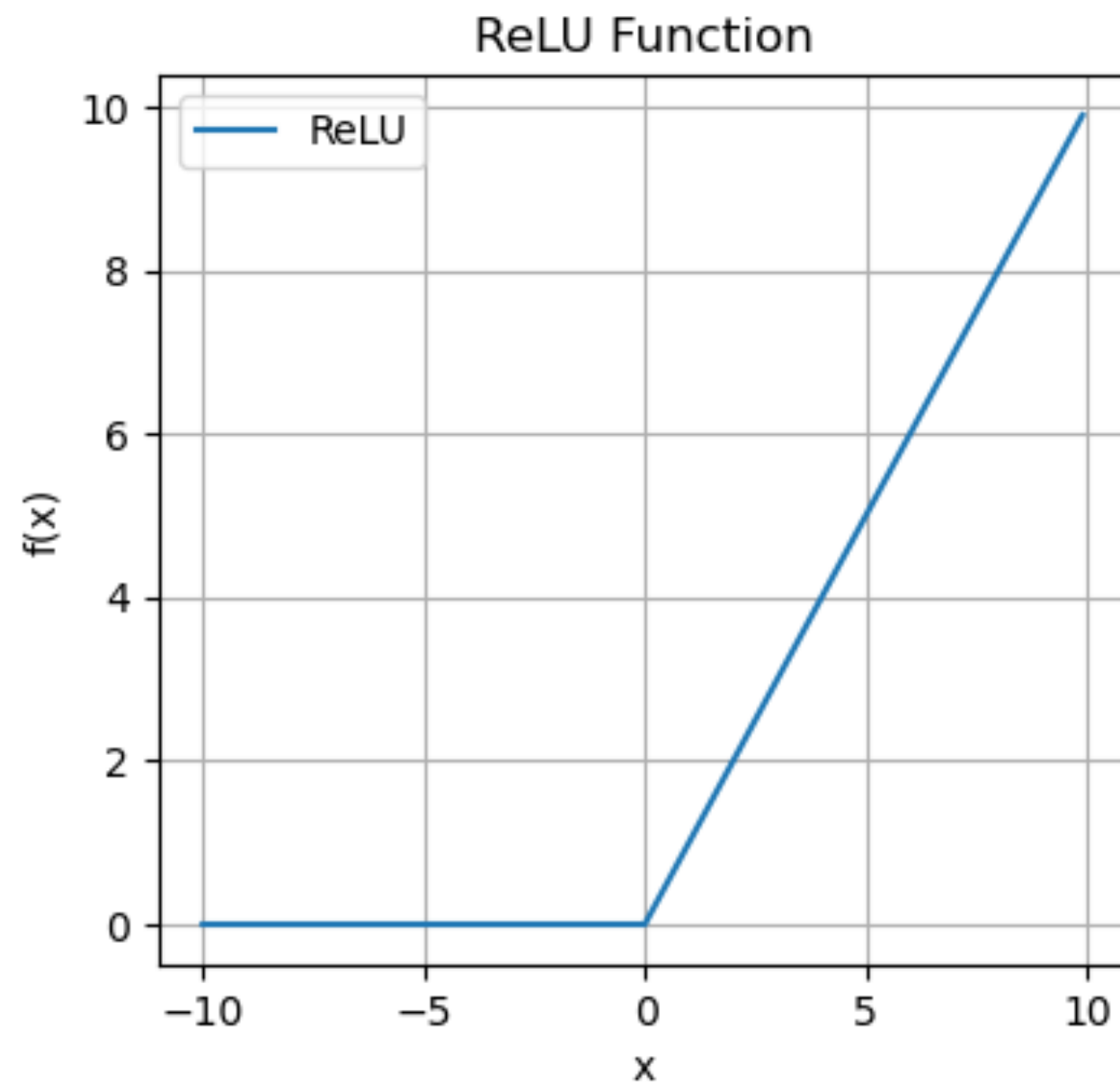
- 인공신경망을 학습할 때 최적화 방법으로 경사 강하법을 사용함
- 모델이 학습 데이터에 대해 발생시킨 에러를 손실 함수로 계산함
- 손실량의 크기에 따라 모든 은닉층에 대해 기울기 계산

기울기 소실(Gradient Vanishing)



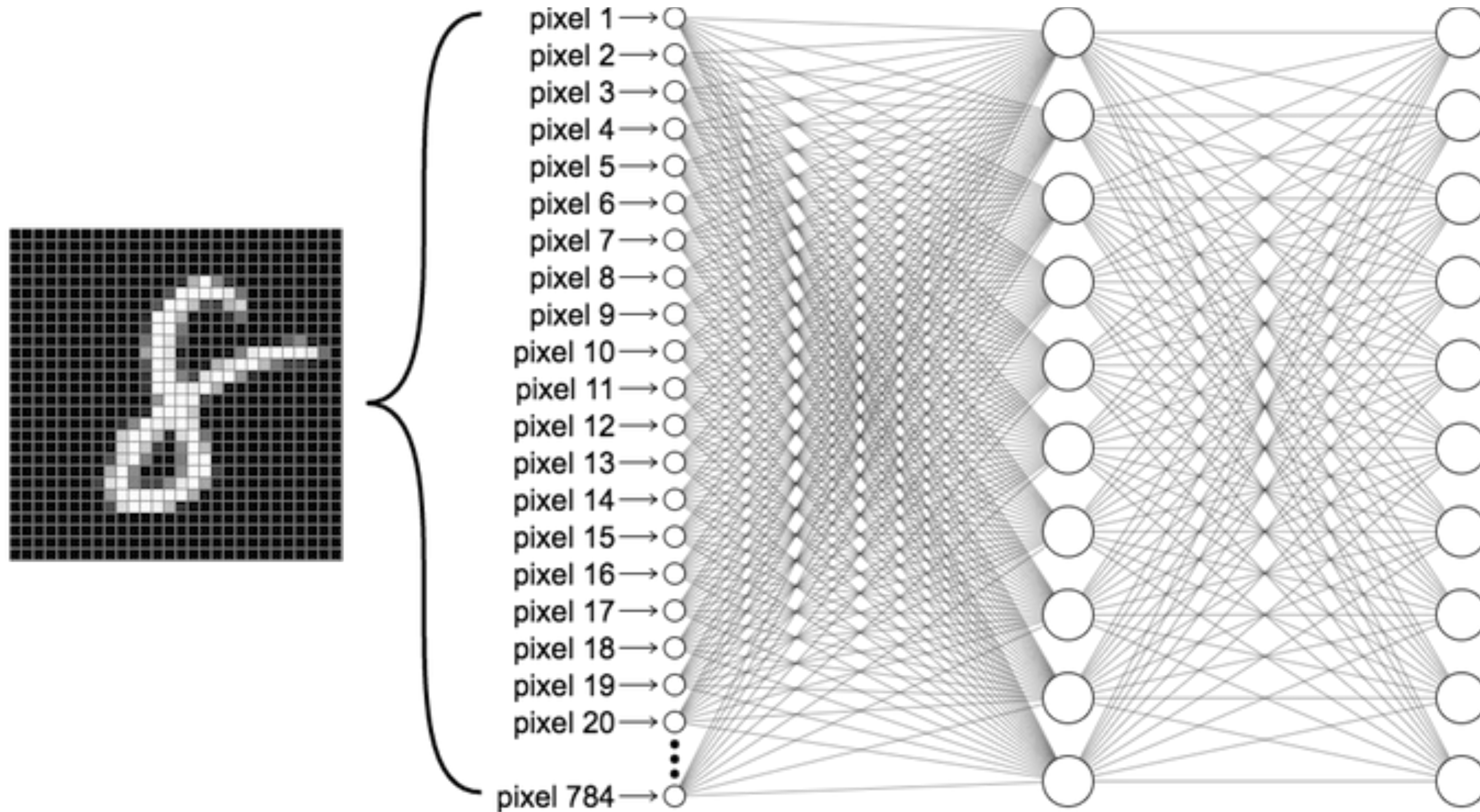
- 손실 함수는 출력층에서 계산됨
- 출력층에서 입력층으로 순차적으로 기울기가 역전파되면서 모든 은닉층의 기울기 계산
- 활성화 함수의 기울기가 매우 작다면, 그 다음 은닉층으로 전달되는 기울기가 0에 수렴하면서 기울기 소실이 발생함

④ 활성화 함수 (3) – Rectified Linear Unit (ReLU)



- 비교적 최신부터 사용된 활성화 함수 (2010 ~)
- 0 이상의 값에 대해 기울기가 1인 동일한 출력을 내보냄
$$f(x) = \max(0, x)$$
- 기울기 소실 문제에서 자유로움
- max 함수는 컴퓨터에서 매우 효율적인 계산이므로, 타 활성화 함수에 비해 빠름

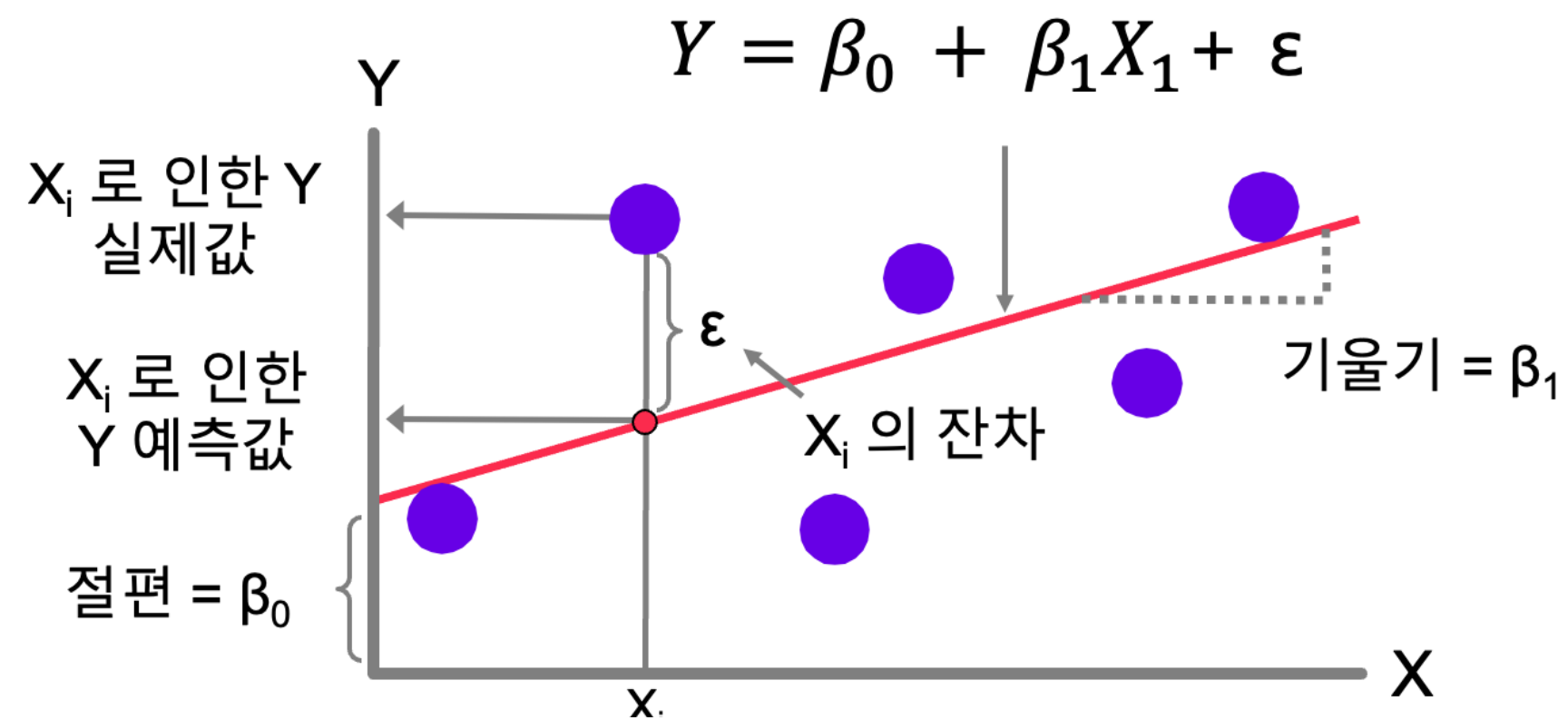
④ MLP 모델 구성 예시



04

원-핫 인코딩

④ 분류 에러의 정의



회귀 문제에서 에러는 정답에서 예측값을 빼면 됨

10년간 수능 번호별 정답 비율

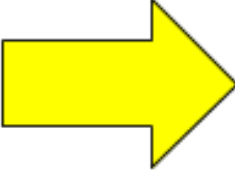


분류 문제에서는 단순 뺄셈으로 에러를 구할 수 없음

ex) 정답 1번인 문제에서 2번보다 5번이 더 틀렸다고 볼 수 없음

④ 원-핫 인코딩(One-Hot Encoding)

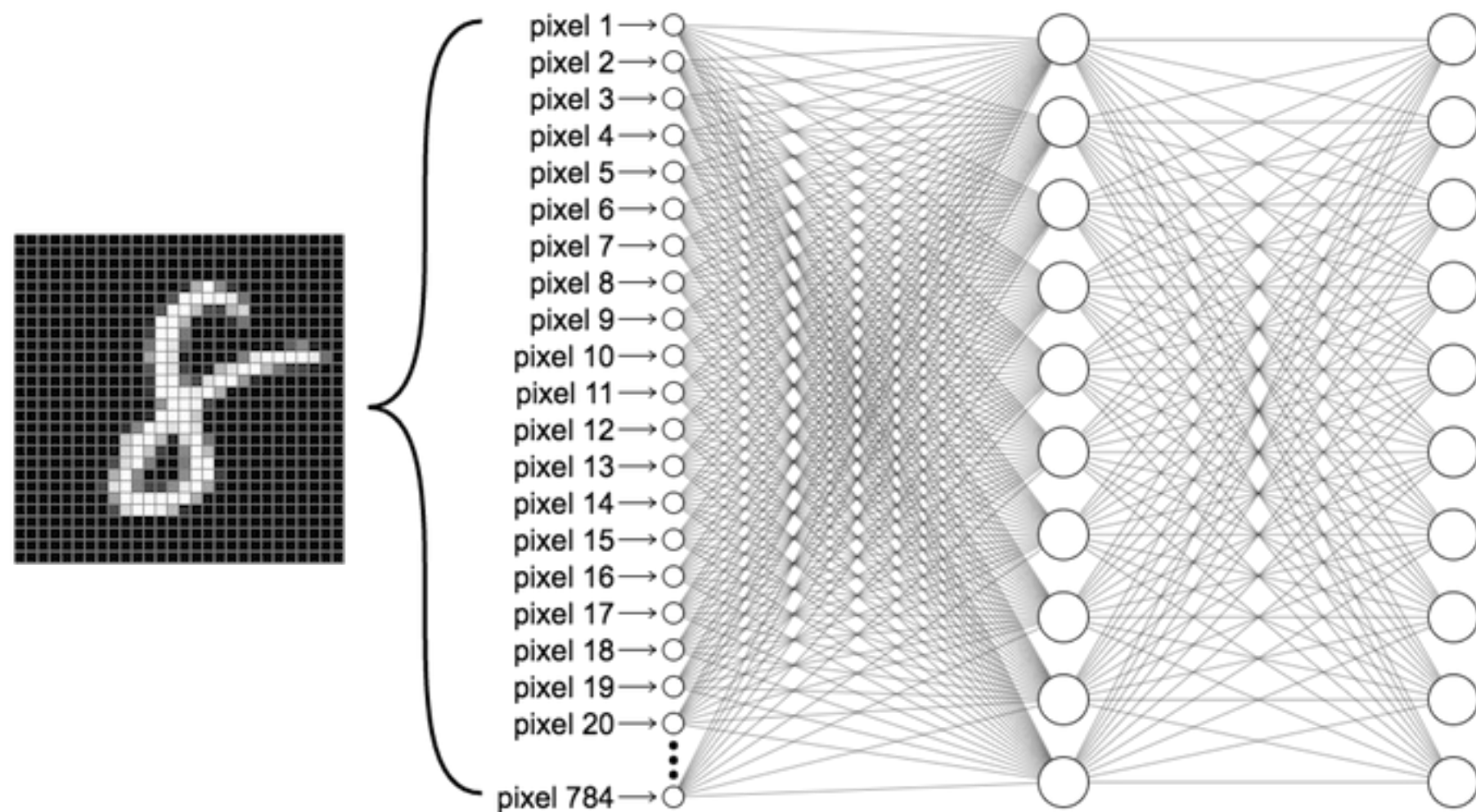
Color	
Red	
Red	
Yellow	
Green	
Yellow	



Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

- 예측하고자 하는 클래스 개수만큼 복제한 뒤, 해당되는 클래스에 1, 나머지에 0을 채워넣는 인코딩 방법
- 종속변수 개수가 증가한 것처럼 취급
- 범주형 데이터 전처리 방법과 동일함

④ 원-핫 인코딩 예시

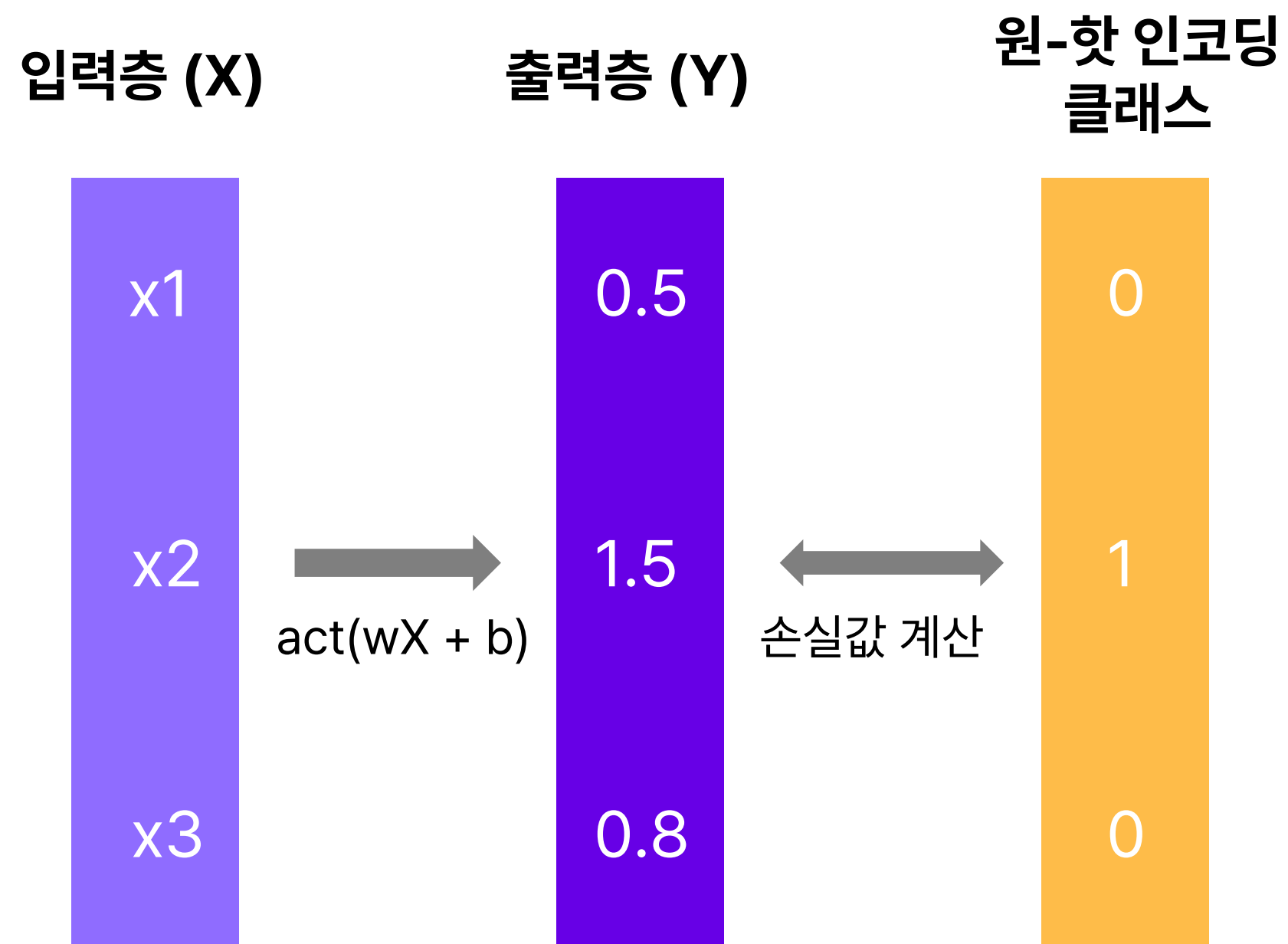


- 손글씨 숫자 분류 문제(MNIST)의 정답 클래스는 0부터 9 까지 총 10 개의 숫자
- 숫자를 직접 예측하지 않고, 원-핫 인코딩으로 정답 데이터 변환
- 신경망의 출력층 차원을 원-핫 인코딩의 크기에 맞게 구성 함

05

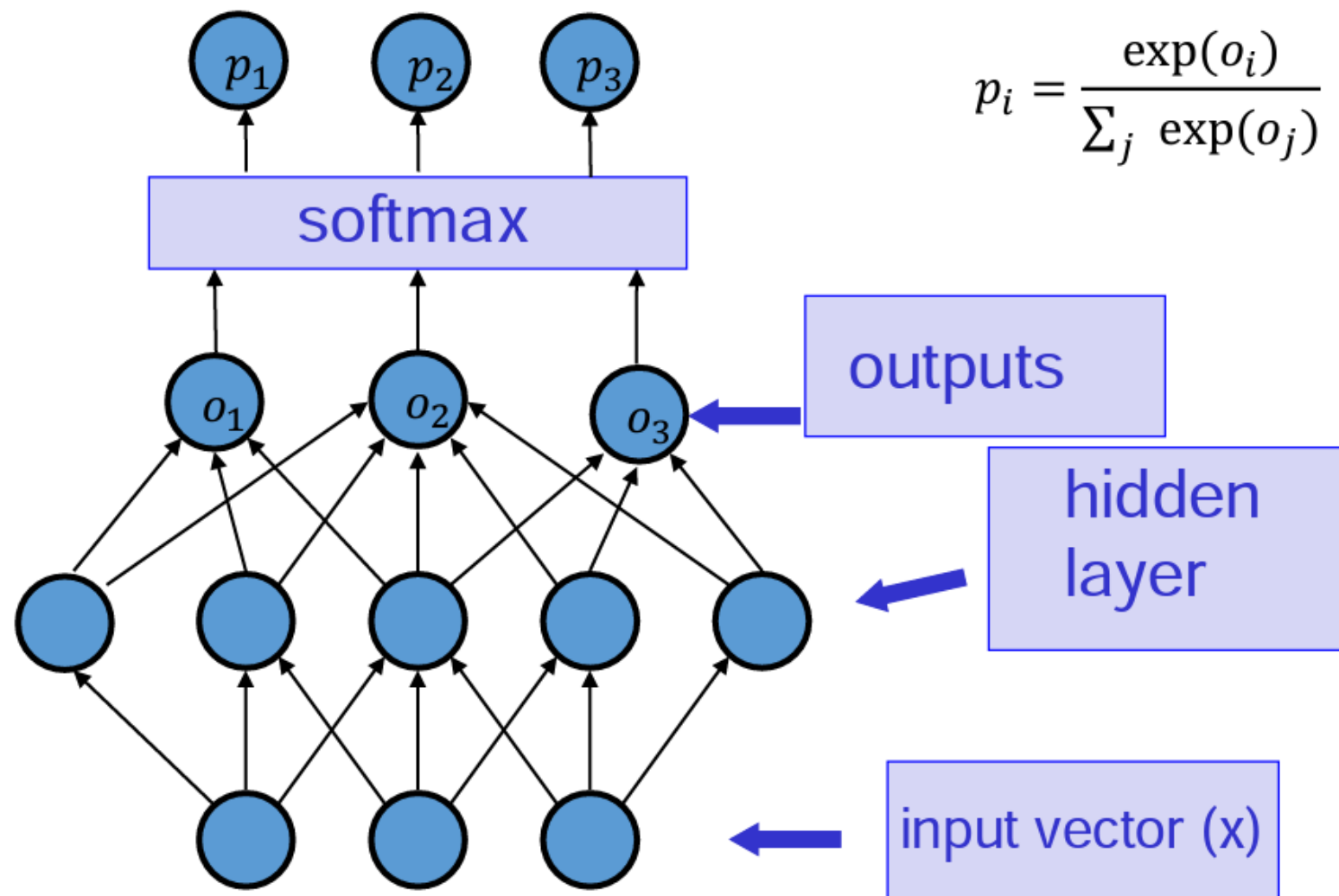
소프트맥스

원-핫 인코딩 후 정답 데이터와의 차이



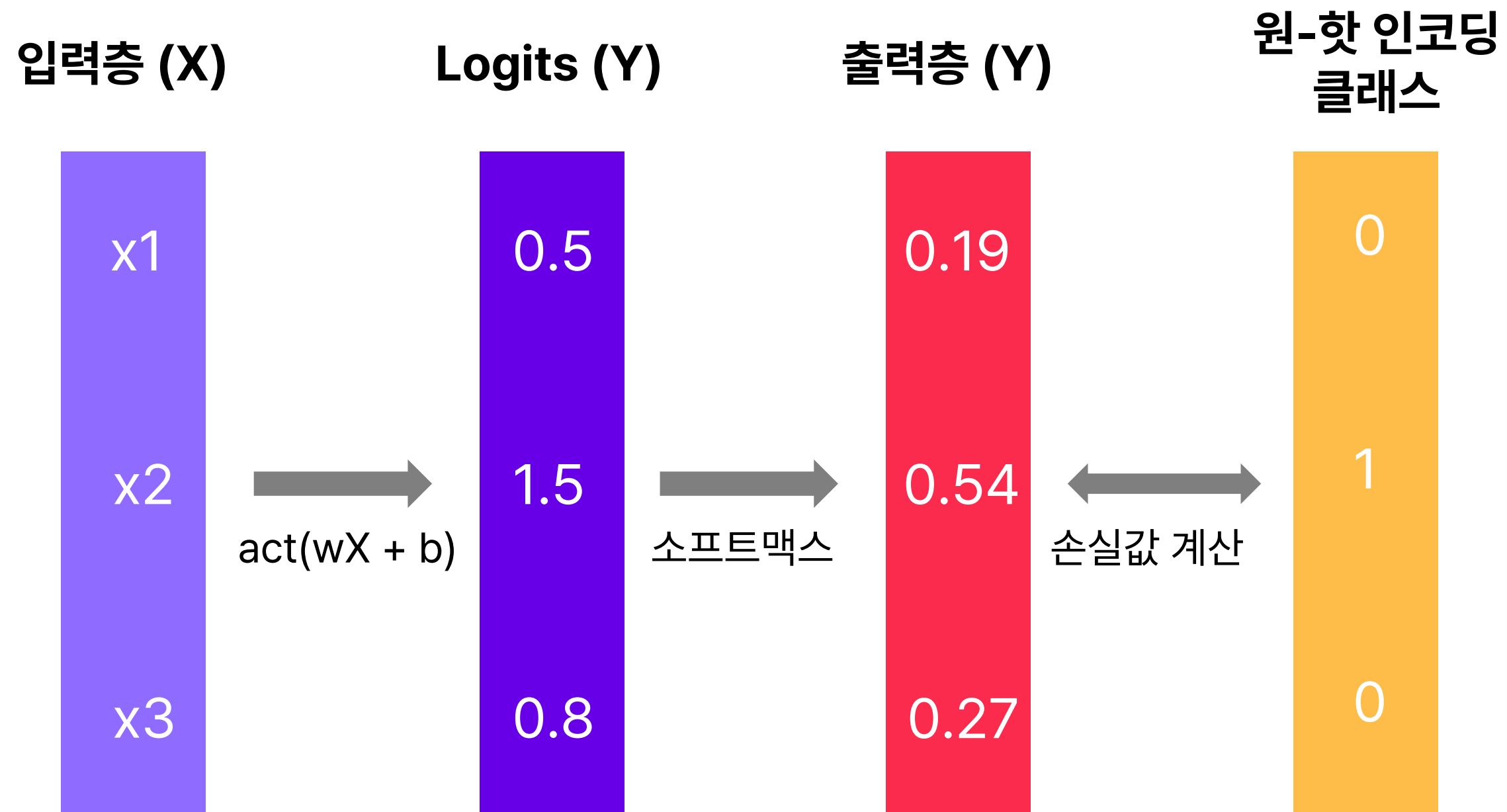
- 모델의 출력층과 원-핫 인코딩 클래스는 일대일 대응이 되는 값을 가짐
- 하지만 두 값을 빼서 손실값을 계산할 수 없음
- 출력층의 값들이 수학적 의미를 갖지 않기 때문
- 출력값들이 각 클래스에 속할 **확률**을 의미하도록 변환해야 함

✔ 소프트맥스(Softmax)



- 출력값 중 최대값을 선택하는 것을 Hardmax 라고 함
- 미분이 가능하도록 해석적 함수를 통해 최대값을 구하는 방법을 소프트맥스(Softmax)라고 함
- 소프트맥스 통과 후 출력값들은 0과 1사이의 값으로 변환되고 합이 1이 됨
- 확률과 동일한 정의

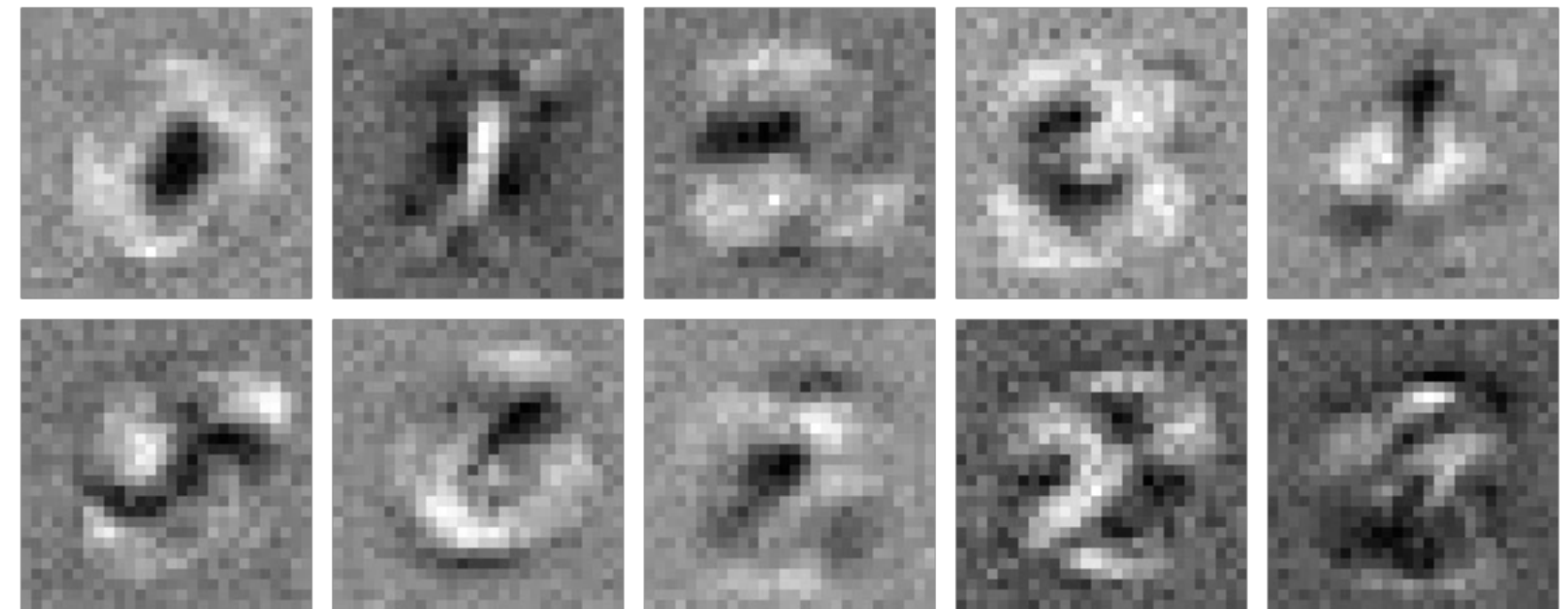
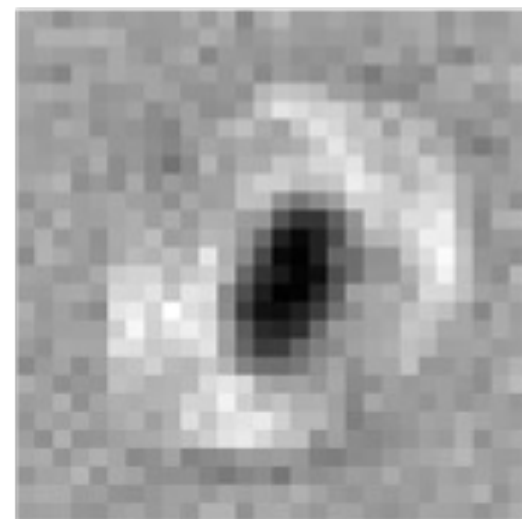
④ 소프트맥스 적용 후



- 소프트맥스 적용 후, 출력값들이 각 클래스에 속하는 확률을 의미하게 됨
- 완벽하게 예측하면 손실값이 0으로 잘 정의됨
- 소프트맥스는 출력층의 활성화 함수로 여겨지기도 함

④ 소프트맥스 적용 후 모델 시각화

W_1	W_2	W_3	...	W_{28}
W_{29}	W_{30}	W_{31}		W_{56}
W_{57}	W_{58}	W_{59}		W_{84}
$\vdots$			$\ddots$	
W_{757}	W_{758}	W_{759}		W_{784}

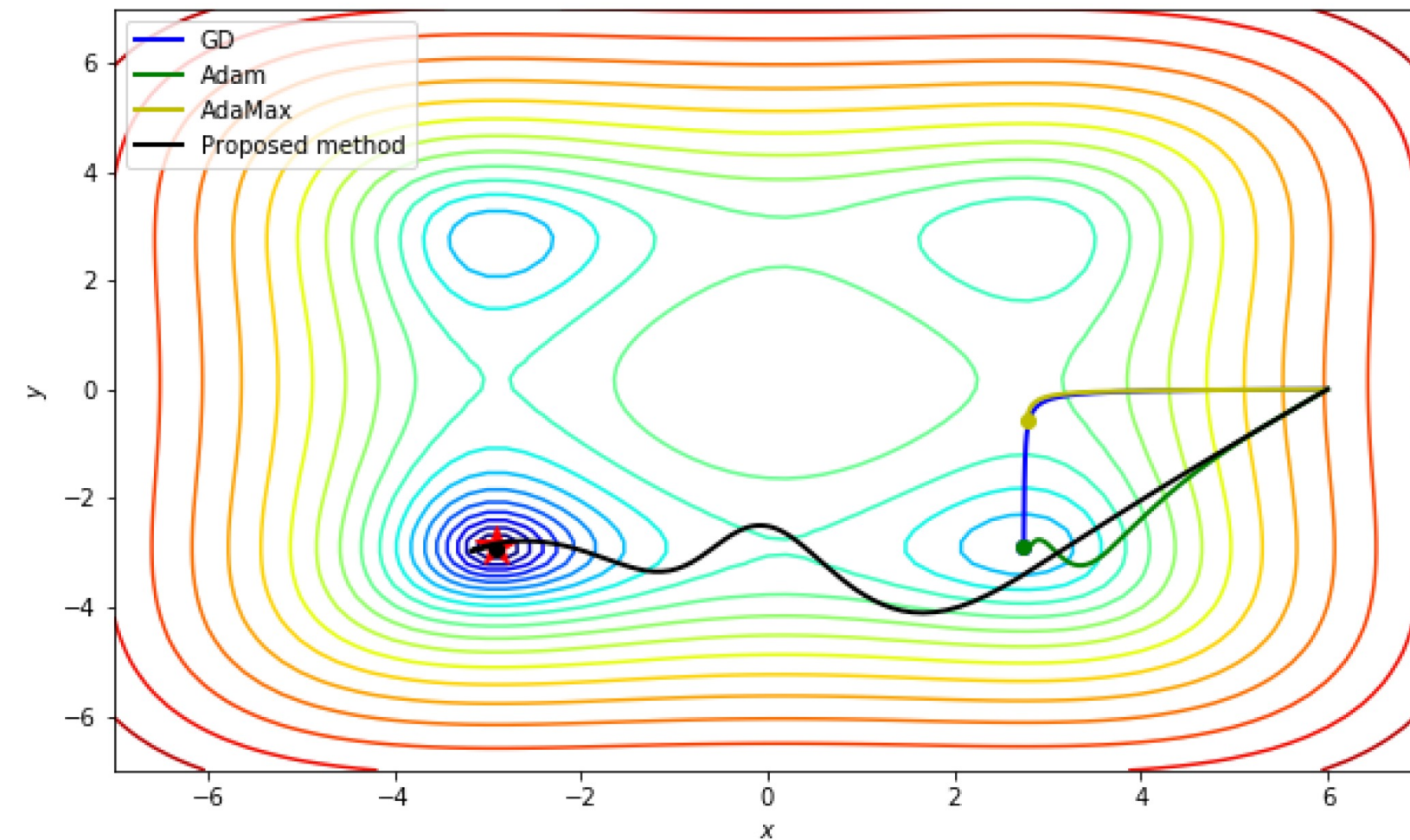
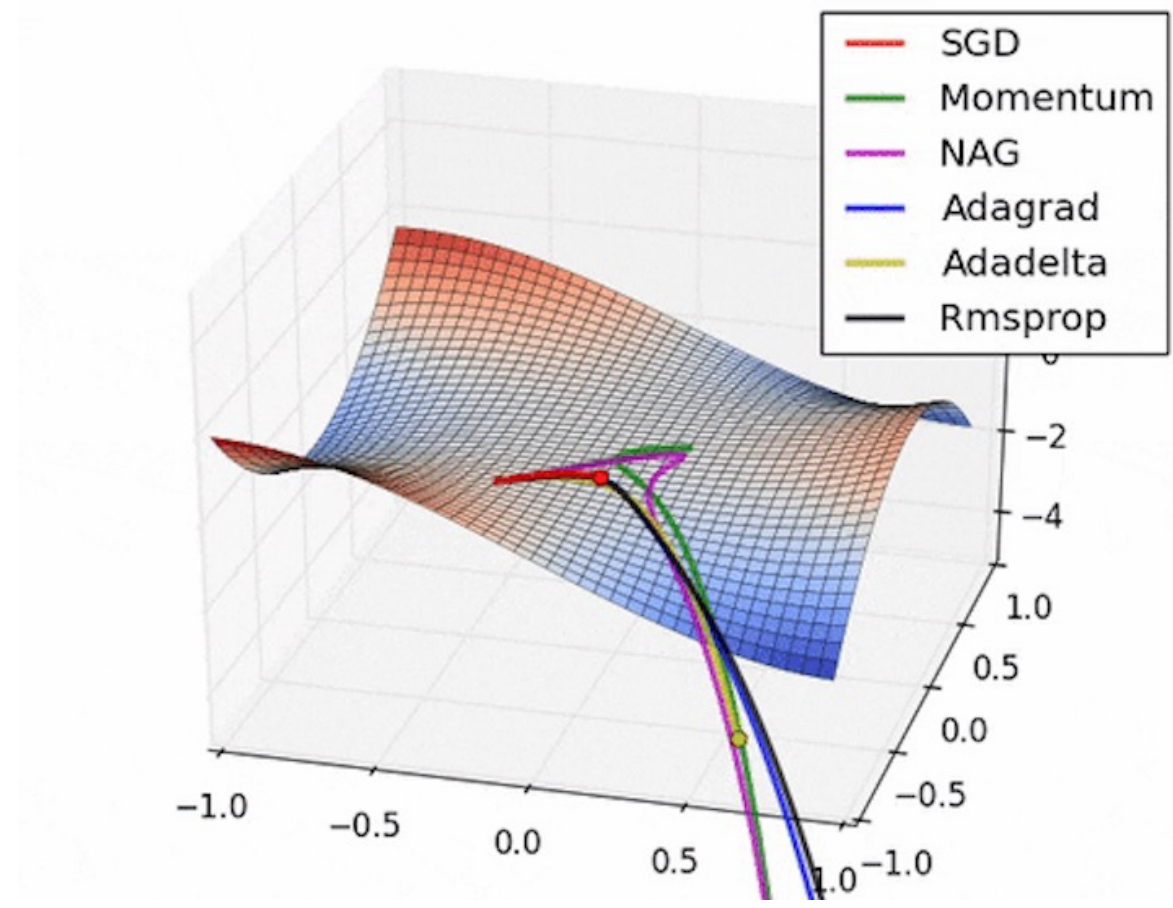


- MNIST를 학습한 모델에서 소프트맥스 통과 후 출력층의 값들을 이미지 형태로 재정렬하여 시각화
- 각 클래스 예측에 사용되는 가중치가 비선형적 패턴을 보여주어, 어떤 픽셀 위치의 값이 해당 숫자 클래스를 예측하는데 더 중요한지를 이해할 수 있음

06

옵티마이저

☑️ 옵티마이저(Optimizer)



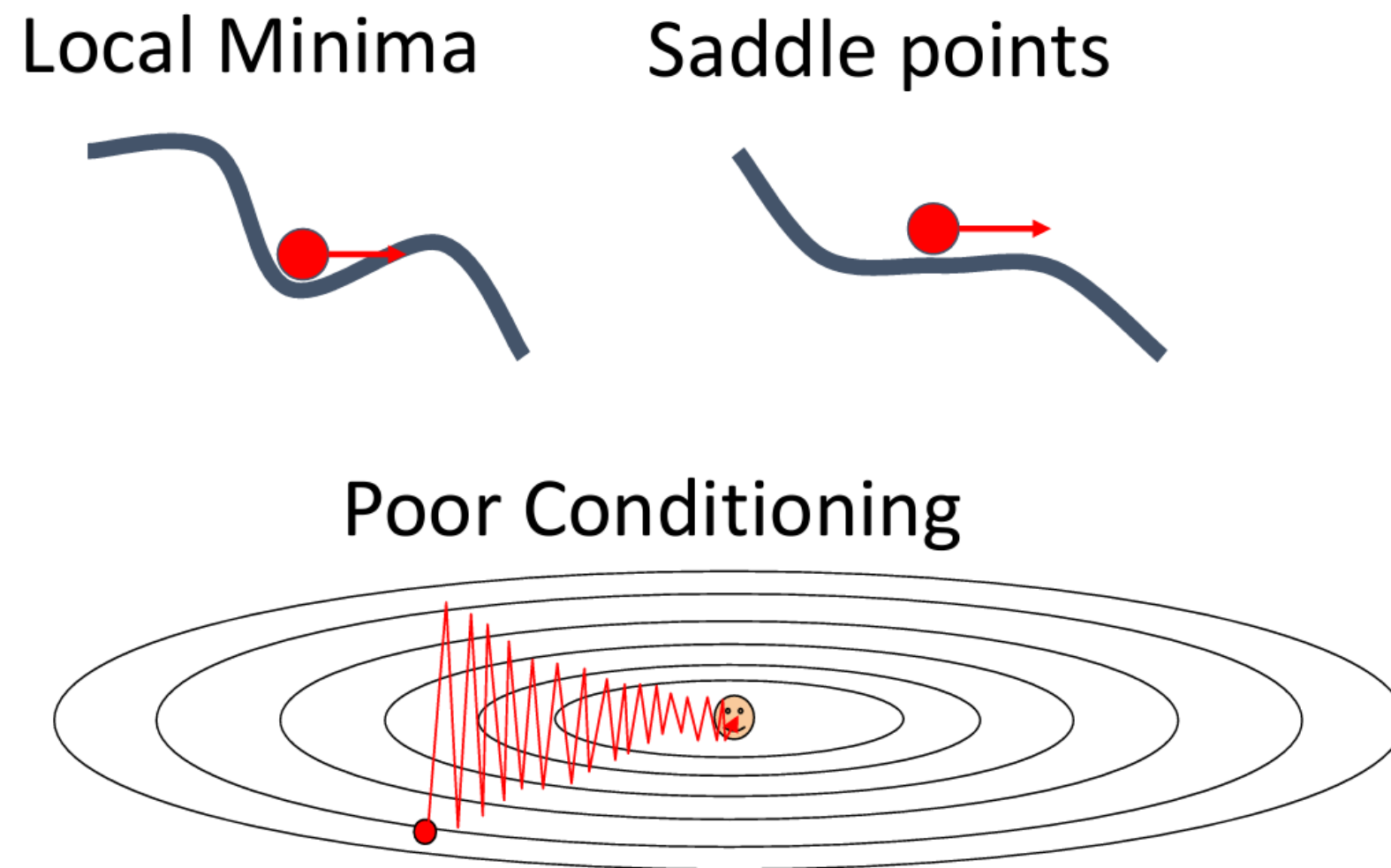
- 인공신경망 모델은 학습데이터를 설정된 에폭(Epoch)만큼 계속 반복하면서 학습을 진행함
- 손실 함수로 계산한 에러를 어떻게 최적화할지 결정하는 함수 – 옵티마이저
- 옵티마이저 설정에 따라 학습 성능의 편차가 매우 클 수 있음

☑️ 옵티마이저 (1) – Stochastic Gradient Descent (SGD)



- 모든 학습데이터에 대해 에러를 구하고 한 번 학습하는 것은 너무 속도가 느림
- 학습데이터 일부만 보고 한 번 학습하여 속도를 빠르게 하자는 아이디어가 SGD
- 이상적인 기울기는 아니라도 **확률적**으로 비슷할 것이라고 가정함
- 학습이 안정적으로 이뤄지지 않음

☑️ 옵티마이저 (2) – Momentum



- 물리적으로 경사면을 따라 강하할 때 중력의 영향을 받아 운동량을 갖게 됨
- 이에 착안하여 운동량을 유지할 수 있도록 유도하여 학습의 안정성을 높인 방법
- 지역 극소값(Local Minima)를 일부 극복할 수 있음

④ 옵티마이저 (3) – Adaptive Gradient (AdaGrad)

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

AdaGrad

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared = decay_rate * grad_squared +
    (1 - decay_rate) * dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

RMSProp

- 한 번 학습할 때 최적화하는 정도를 설정하는 학습률(Learning Rate)
- 기울기마다 학습률을 다르게 적용해서 더욱 안정적으로 최적화 – AdaGrad & RMSProp
- 기울기가 작은 " 평평한 " 구간은 학습이 가속, 기울기가 큰 " 험한 " 구간은 학습이 감속

☑️ 옵티마이저 (4) – AdaM

for input to the CNN; each colored pixel in the image yields a 7D one-hot vector. Following common practice, the network is trained end-to-end using stochastic gradient descent with the Adam optimizer [22]. We anneal the learning rate to 0 using a half cosine schedule without restarts [28].

Bakhtin, van der Maaten, Johnson, Gustafson, and Girshick, NeurIPS 2019

We train all models using Adam [23] with learning rate 10^{-4} and batch size 32 for 1 million iterations; training takes about 3 days on a single Tesla P100. For each mini-batch we first update f , then update D_{img} and D_{obj} .

Johnson, Gupta, and Fei-Fei, CVPR 2018

ganized into three residual blocks. We train for 25 epochs using Adam [27] with learning rate 10^{-4} and 32 images per batch on 8 Tesla V100 GPUs. We set the cubify thresh-

Gkioxari, Malik, and Johnson, ICCV 2019

sampled with each bit drawn uniformly at random. For gradient descent, we use Adam [29] with a learning rate of 10^{-3} and default hyperparameters. All models are trained with batch size 12. Models are trained for 200 epochs, or 400 epochs if being trained on multiple noise layers.

Zhu, Kaplan, Johnson, and Fei-Fei, ECCV 2018

- RMSProp과 Momentum 방법을 통합한 것이 AdaM 옵티마이저
- 가장 흔히 사용되고, 고민없이 선택해도 무방한 방법
- 대부분의 최신 인공지능 모델이 AdaM으로 학습됨

인공신경망 학습 과정

